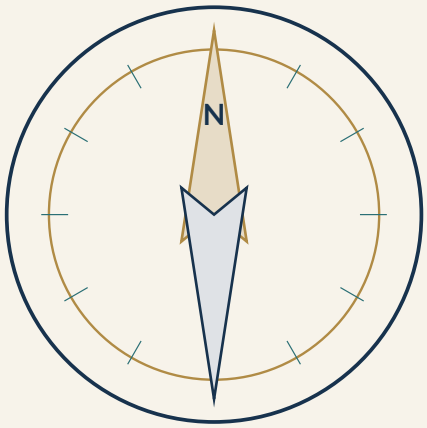
TALL TALE STUDIO

VISION WAYPOINT
& EXTERNAL AR
GOVERNING SPECIFICATION



Source of Truth for Studio Authoring, Capture, Visual Verification, Runtime Presentation, Testing, Safety, and Phased Implementation

Document ID:	TT-VISION-GOV-001
Version:	1.0
Status:	Governing Baseline
Date:	July 17, 2026
Implementation State:	Phase A complete; Phase B authorized for incremental implementation
Primary Use:	Codex implementation source of truth and creator-product specification

Document Authority and Intended Use

Governing Rule

This document is the controlling product and engineering specification for the Tall Tale Studio visual verification and external augmented-reality system. Codex, human developers, designers, testers, and future contributors MUST use it as the primary source of truth. Implementation details may improve, but product behavior, safety boundaries, data ownership, workflow expectations, verification philosophy, and release gates MUST NOT be silently changed.

This specification consolidates the complete design developed for:

- passive cinematic scene transformations;
- active screen-based location verification;
- highly accurate multi-model visual localization;
- reusable Studio-authored **Vision Waypoints**;
- a local Capture Companion that observes only the selected game window;
- versioned waypoint packages that can be built once and reused without custom coding;
- player-friendly verification that does not expose computer-vision machinery;
- Captain controls, diagnostics, shadow testing, and manual fallback;
- privacy, anti-cheat, reliability, calibration, publishing, and long-term maintainability.

The governing intent is to prevent implementation drift. A future developer must not replace this architecture with a one-image classifier merely because it is easier, nor embed one-off scripts inside individual Tall Tales, nor require creators to manipulate training folders, command lines, or Python notebooks. The product is a **creator tool**, not a collection of clever prototypes held together by Codex prompts and optimism.

Normative keywords

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative:

- **MUST / MUST NOT**: mandatory for conformance.
- **SHOULD / SHOULD NOT**: expected unless a documented architectural decision explains the exception.
- **MAY**: optional behavior that must not conflict with mandatory requirements.

Change control

Any material departure from this specification requires an Architecture Decision Record, or ADR, containing:

1. the requirement being changed;
2. the reason the existing requirement is inadequate;
3. alternatives considered;
4. effects on creator workflow, player experience, privacy, performance, safety, and compatibility;
5. migration and rollback plans;
6. explicit approval before merging.

A library substitution that preserves all governing behavior does not require a product-specification rewrite, but it SHOULD receive a technical ADR. For example, replacing SALAD with a better place-recognition model is acceptable if the hard-negative, ambiguity, calibration, and package contracts remain intact. Replacing hierarchical verification with a single opaque confidence score is not acceptable.

Document Control

Field	Governing value
Project	Tall Tale Studio / Date Nights
Capability	Vision Waypoints and external augmented reality
Baseline	Version 1.0
Owner	Project Captain / Studio product owner
Primary implementer	Codex-assisted development under human review
Current phase	Passive presentation complete; guided active verification next
Primary platform	Windows creator/player Companion plus browser-based Studio and Player surfaces
Initial game integration	Sea of Thieves, read-only visual observation only
Reliability posture	False-positive-averse, ambiguity-rejecting, guided-retry design
Privacy posture	Local capture and inference by default; no retained footage by default
Modification boundary	No game injection, memory reading, file modification, automation, or in-game overlay

How to Read This Specification

Readers implementing one piece at a time should use the following order:

1. Read Chapters 1 through 7 to understand product intent, terminology, roles, architecture, and non-negotiable rules.
2. Read Chapters 8 through 16 before implementing Studio authoring or the Capture Companion.
3. Read Chapters 17 through 25 before implementing any automatic verification decision.
4. Read Chapters 26 through 31 before publishing waypoints or enabling automatic story progression.
5. Read Chapters 32 through 39 before adding cloud services, sharing, live AR, or model training.
6. Use Chapters 40 through 44 and the appendices as Codex work-package boundaries, schemas, checklists, and acceptance tests.

Contents

Chapter 1

Executive Summary

The Tall Tale companion will support story moments in which the real Sea of Thieves view transitions into a modified, cinematic, or interactive interpretation. A player may reach a rock and see a faint message appear carved into it, hold an ordinary mermaid gem and watch it transform into the story's true artifact, discover a hidden symbol, or trigger a journal reveal only after reaching and inspecting the correct place.

Phase A provides the passive presentation framework: the story system can trigger prepared images, videos, audio, Three.js scenes, or transitions after a manual Captain action or another trusted event. Phase B adds player-initiated automatic verification. The player holds an **Inspect Surroundings** control for several seconds while slowly scanning the scene. A local Companion captures a short burst from the selected Sea of Thieves window, discards unusable frames, and proves the location through multiple independent layers:

1. current story-stage restriction;
2. capture-quality validation;
3. global visual place retrieval;
4. comparison against known similar wrong places;
5. local feature matching;
6. geometric verification;
7. camera-pose localization within a small 3D reconstruction;
8. spatial distribution of evidence;
9. consistency across several frames;
10. checkpoint-specific rules.

The verifier MUST never unlock from one screenshot, one model, or one weighted confidence score. A wrong result can destroy the story, so ambiguity is rejected. At the same time, legitimate users must not be punished by brittle exact-image matching. The system therefore localizes a new camera view inside a reconstructed target scene and provides guided retries when evidence is incomplete.

Most importantly, the entire system must be reusable. A creator must be able to build a **Vision Waypoint** inside the Studio through a guided wizard. The creator records the correct location, walks the valid player region, records confusing wrong locations, confirms important visual regions, runs positive and negative tests, and publishes a versioned package. The Studio and Companion perform frame extraction, mapping, feature generation, calibration, testing, and packaging automatically. Codex builds the tool once; creators use it repeatedly.

Chapter 2

Product Vision and Core Objectives

2.1 Product vision

The player should feel that the Tall Tale companion is not merely waiting for a button press. It should appear to perceive the journey, recognize meaningful places and objects, and reveal a hidden narrative layer at exactly the right moment.

The creator should feel as though they are using a polished game editor. Creating a visually verified story event should be comparable to creating a checkpoint, adding a cinematic, or placing a trigger in a modern level editor. It must not feel like training a machine-learning model.

2.2 Primary objectives

The system MUST:

- enable automatic verification of story-critical game locations and visual conditions;
- minimize false positives strongly enough that a wrong location does not prematurely trigger a reveal;
- avoid exact-view brittleness by supporting reasonable variation in player position, angle, distance, field of view, resolution, lighting, weather, and graphical settings;
- guide legitimate players toward successful verification instead of returning generic failures;
- allow nontechnical creators to author, test, reuse, version, and share Vision Waypoints in the Studio;
- keep the Sea of Thieves process untouched;
- perform runtime processing locally by default;
- attach verification results to a general event-driven story engine rather than hard-code cinematics into the verifier;
- preserve Captain oversight and manual recovery;
- allow gradual promotion from development to shadow mode, Captain-confirmed mode, and fully automatic progression;
- support future live external-AR effects without requiring the core authoring system to be rebuilt.

2.3 Quality hierarchy

When tradeoffs occur, the governing priority is:

1. prevent false story progression;
2. preserve player trust and privacy;
3. provide a successful guided retry for legitimate players;
4. keep creator authoring simple;
5. maintain explainability and diagnostics;
6. optimize performance;
7. add spectacle.

A beautiful ten-second reveal is valuable. A beautiful ten-second reveal triggered by the wrong palm tree is accidental comedy and therefore not a release criterion.

Chapter 3

Governing Principles

3.1 The verifier proves; it does not guess

The system MUST use mandatory evidence gates. A high score from one stage cannot compensate for a failed geometric, pose, negative-margin, or temporal gate.

3.2 Current story context narrows the problem

The system MUST inspect only the currently active waypoint and its explicit confusers. It is not required to classify the entire Sea of Thieves world continuously.

3.3 Multi-frame evidence is the default

Phase B MUST use a short guided scan, not a single screenshot. Multiple frames provide view diversity, obstruction recovery, motion evidence, and sequence consistency.

3.4 Geometry outranks appearance

Global similarity may retrieve candidates, but geometric evidence and camera localization MUST determine whether the physical scene is consistent with the target.

3.5 Hard negatives are first-class data

Creators MUST record similar wrong places for high-reliability waypoints. The system MUST compare target confidence against known confusers and refuse close ties.

3.6 Ambiguity is a valid result

The internal outcome set is:

- `VERIFIED`;
- `INSUFFICIENT_VISUAL_EVIDENCE`;
- `NOT_AT_TARGET`.

The verifier MUST NOT force every attempt into success or failure when the evidence is ambiguous.

3.7 Story logic is separate from recognition logic

A Vision Waypoint proves a condition. The story graph decides what happens next. The same waypoint may trigger different outcomes in different Tall Tales.

3.8 The creator sees guidance, not computer-vision jargon

The default Studio experience MUST use language such as “record more of the right side,” “capture another similar rock,” and “move farther away.” Raw model thresholds belong in engineering diagnostics.

3.9 Published behavior is versioned

A published Tall Tale MUST reference an immutable waypoint version. Updating a library waypoint MUST NOT silently alter a published story.

3.10 Safety boundaries are architectural, not optional settings

The Companion MUST NOT inject into the game, read process memory, alter game files, simulate input, or render an in-game overlay. These are prohibited implementation paths, not merely disabled defaults.

Chapter 4

Scope and Non-Goals

4.1 In scope

This specification covers:

- passive image and video reveals;
- view-matched pre-rendered reveals;
- external augmented-reality presentation in the companion surface;
- creator-authored visual location verification;
- exact landmarks, area arrival, viewpoints, object inspection, held-item conditions, and sequences;
- local screen capture of a user-selected game window;
- local and optional cloud-assisted waypoint building;
- global place recognition, hard-negative comparison, feature matching, 3D reconstruction, camera-pose localization, temporal consistency, and checkpoint-specific rules;
- Studio UX, Companion UX, Player UX, Captain controls, diagnostics, packages, schemas, testing, publishing, and lifecycle management.

4.2 Explicitly out of scope for the governing baseline

The baseline MUST NOT depend on:

- game memory inspection;
- modification of Sea of Thieves files or rendering pipeline;
- code injection, DLL hooks, ReShade, or an in-game overlay;
- automated player movement, mouse control, key simulation, or gameplay automation;
- competitive intelligence, enemy detection, loot highlighting, fog removal, or navigation assistance unrelated to the authored story;
- continuous global recognition of every island and object;
- mandatory custom model training per waypoint;
- mandatory cloud upload of gameplay footage;
- public marketplace moderation infrastructure in the first implementation phase.

These exclusions protect safety, product focus, and the player's account. Humans have already invented enough ways for software to become unexpectedly bannable.

Chapter 5

Terminology

Term	Governing definition
Vision Waypoint	A reusable, versioned Studio asset that proves a visual location, viewpoint, object, item, area, or sequence condition.
Checkpoint	A runtime instance of a Vision Waypoint attached to a story stage.
Target recording	Creator footage captured at the correct location and acceptable viewpoints.
Hard negative	A deliberately recorded wrong place or condition that closely resembles the target.
Borderline positive	A valid but difficult viewpoint near the edge of acceptance.
Accepted pose volume	The approved 3D camera positions and viewing orientations from which verification may succeed.
Stable region	A fixed visual area suitable for matching, such as rock cracks or architecture.
Ignored region	Dynamic or unreliable imagery such as sky, water, particles, players, HUD, or foliage.
Global descriptor	A compact representation used to retrieve visually similar reference views.
Local feature	A distinctive point or semi-dense correspondence used to verify detailed geometry.
Inlier	A feature match consistent with the estimated geometric transformation.
Reprojection error	The disagreement between observed image points and points predicted by the estimated camera geometry.
Sparse 3D map	A Structure-from-Motion reconstruction of stable visual landmarks and reference camera poses.

Term	Governing definition
Shadow mode	Verification runs and records what it would have done but cannot advance the story.
Captain-confirmed mode	Automatic evidence is shown to the Captain, who approves progression.
Automatic mode	A certified waypoint advances the story directly after all mandatory gates pass.
External AR	A modified representation shown in the companion surface, separate from and not injected into the game.
Vision Build Engine	The subsystem that curates recordings, builds maps and descriptors, calibrates thresholds, evaluates tests, and produces packages.
Capture Companion	The local Windows application that captures the selected game window and performs authoring and runtime operations.
Story-Critical profile	The strictest standard preset for events where a false positive would materially damage the narrative.

Chapter 6

Users, Roles, and Operating Modes

6.1 Creator

The Creator builds Tall Tales and Vision Waypoints. A Creator MUST be able to complete the normal workflow without command-line tools, source code, model terminology, or direct filesystem manipulation.

6.2 Captain

The Captain operates a live Tall Tale, monitors exceptional verification attempts, receives optional notifications, and may approve, reject, or manually trigger an event. The Captain may also be the Creator, but the roles remain separate in the product model.

6.3 Player

The Player sees only story-appropriate instructions and feedback. The Player SHOULD NOT see match counts, thresholds, coordinates, or model names.

6.4 Studio modes

6.4.1 Guided mode

Default mode for ordinary creators. The system chooses the recognition strategy and thresholds, provides capture instructions, and blocks unsafe publication.

6.4.2 Detailed mode

Allows region masking, pose-volume editing, required landmarks, strictness controls, test-case management, and deeper reliability review.

6.4.3 Engineering mode

Allows model selection, feature visualizations, match inliers, pose estimates, score distributions, calibration curves, and runtime profiling. This mode exists because some people see an “Advanced Diagnostics” disclosure and regard it as a personal invitation.

6.5 Runtime modes

Mode	Behavior
Manual	Captain triggers the presentation without automatic vision.
Shadow	Vision evaluates attempts but cannot progress the story.
Captain-confirmed	Vision proposes a result; Captain approves progression.
Automatic	Certified verification progresses the story directly.
Disabled	Waypoint is retained in the story graph but not currently available.

Chapter 7

System Architecture

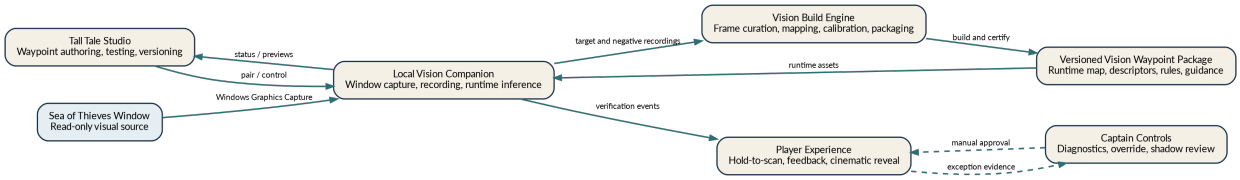


Figure 7.1: System context and data flow

7.1 Major components

7.1.1 Tall Tale Studio

The browser-based Studio owns:

- waypoint library and metadata;
- creation wizard;
- target and negative recording orchestration;
- coverage visualization;
- accepted-pose editor;
- stable-region and ignore-region editing;
- reliability dashboards;
- test management;
- versioning and publishing;
- story-block attachment;
- creator permissions and sharing scope.

7.1.2 Local Vision Companion

The local Companion owns:

- window selection and secure screen capture;
- pairing with the Studio and Player web surfaces;
- authoring recording sessions;
- local frame buffering;
- quality checks and live capture guidance;
- local runtime inference;

- hardware capability detection;
- local package cache;
- privacy controls and capture indicators;
- diagnostic evidence collection when authorized.

7.1.3 Vision Build Engine

The Build Engine owns:

- frame extraction;
- blur and exposure rejection;
- near-duplicate removal;
- visual diversity analysis;
- descriptor and feature generation;
- sparse scene reconstruction;
- accepted-region estimation;
- negative indexing;
- threshold calibration;
- automated offline testing;
- package assembly and signing;
- build reports and actionable remediation.

The Build Engine MAY run locally or in a cloud-assisted environment. The contracts and outputs MUST remain identical.

7.1.4 Vision Waypoint Package

A package is the immutable runtime artifact containing:

- manifest and schema version;
- thumbnail and creator-facing metadata;
- selected reference descriptors;
- local features and 3D map data;
- stable and ignored region masks;
- hard-negative index;
- accepted pose volume;
- verification rules and thresholds;
- player guidance strings;
- reliability certification metadata;
- package integrity information.

Raw creator recordings MUST NOT be required at runtime.

7.1.5 Player web experience

The Player surface owns:

- story instructions;
- scan initiation and progress;
- feedback messages;
- event-driven presentation;

- journal and history updates;
- fallback messaging when the Companion is absent or disabled.

7.1.6 Captain controls

The Captain surface owns:

- verification attempt queue;
- current checkpoint status;
- evidence summary;
- manual approve, reject, and trigger actions;
- mode promotion and demotion where permitted;
- privacy-conscious diagnostics;
- live recovery.

Chapter 8

Phased Product Roadmap

8.1 Phase A: Passive cinematic framework - completed baseline

Phase A provides event-driven presentation without automatic visual proof. It includes prepared images, videos, audio, journal reveals, and transitions initiated manually or by trusted story events.

8.2 Phase B: Guided visual verification - current target

Phase B introduces:

- Companion pairing;
- hold-to-scan capture;
- one pilot waypoint;
- frame-quality filtering;
- multi-layer visual verification;
- three-state outcomes;
- guided retries;
- shadow mode;
- Captain fallback;
- Studio creation and testing flow.

8.3 Phase C: Automatic stage-aware verification

After Phase B demonstrates reliability, certified checkpoints MAY listen at a low rate while active, or automatically prompt when a likely scene is present. Automatic background verification MUST remain stage-aware and privacy-visible.

8.4 Phase D: Live external AR

Phase D adds:

- perspective-anchored carved messages;
- tracked symbols and runes;
- ghostly figures or overlays in the companion representation;
- live or semi-live item transformation;

- optical-flow-driven motion;
- richer 3D artifact rendering;
- view-conditioned effects.

Phase D MUST build on the same Vision Waypoint packages and events. It must not require re-authoring the core location proof.

Chapter 9

Phase A Presentation Framework

9.1 Governing presentation model

The presentation engine receives trusted story events and performs a sequence of visual and audio actions. Presentation assets are independent of how the event was verified.

9.2 Presentation levels

9.2.1 Level 1: pre-rendered transition

The system fades from the current companion interface or a captured frame into a prepared image or video. This is the most reliable and artistically controlled method.

A carved-rock event may:

1. freeze or softly blur the current captured view;
2. crossfade to a matched edited frame;
3. reveal faint cuts or glowing letters over several seconds;
4. add particles, dust, reflected light, or sound;
5. reveal a Captain message or journal entry;
6. return to the story interface.

9.2.2 Level 2: view-matched reveal

The waypoint stores several edited or rendered variants. Runtime localization chooses the asset closest to the player's actual camera pose, distance, and pitch.

A selection key may include:

- left, center, or right viewpoint;
- close, medium, or far distance;
- upward, level, or downward pitch;
- day, night, or low-light variant.

9.2.3 Level 3: external augmented reality

The system locates a surface in the captured frame, calculates a perspective transform, and renders an effect into the companion view. The game itself remains untouched.

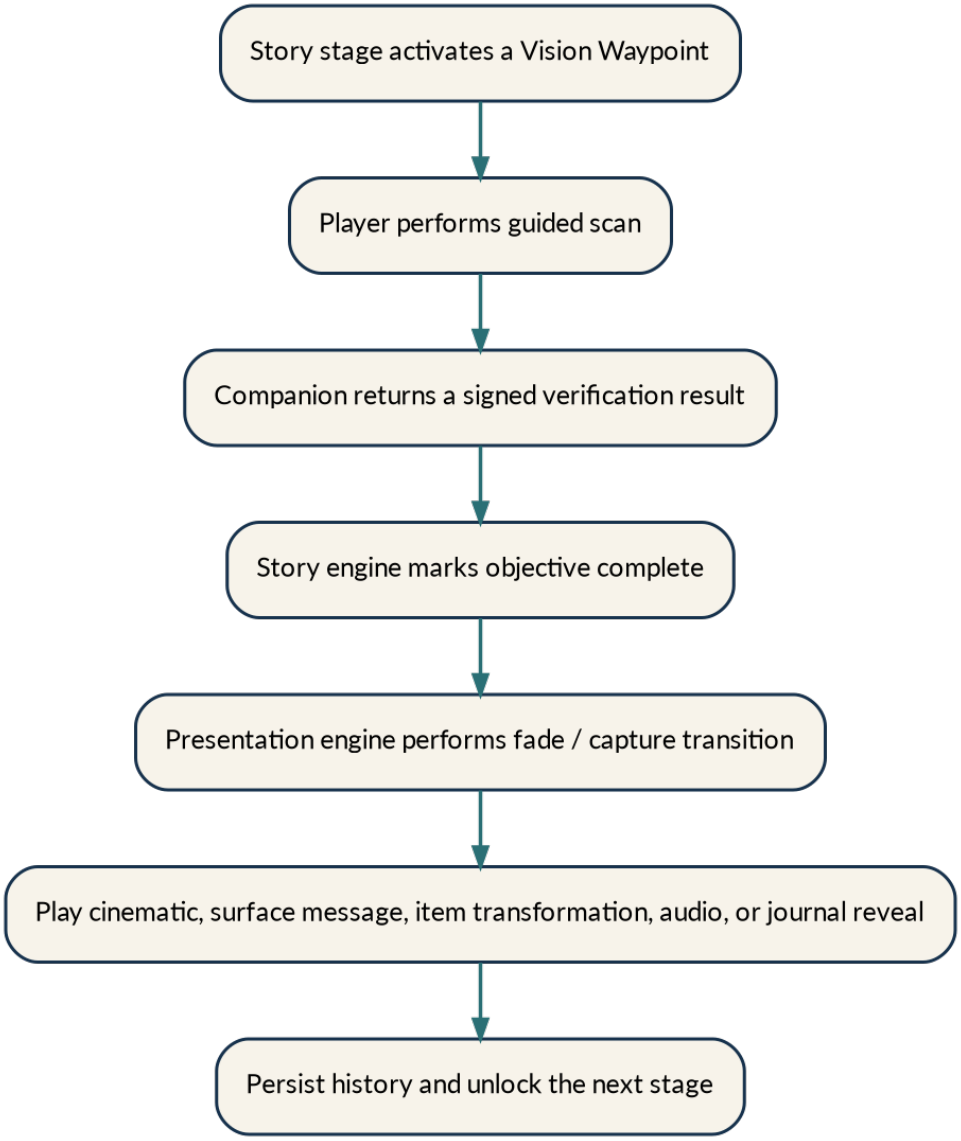


Figure 9.1: Story verification and presentation event flow

Possible effects include:

- a message gradually appearing in rock cracks;
- wet, glowing, burned, or dust-filled writing;
- symbols visible only from the correct angle;
- hidden objects or spectral figures;
- a texture that tracks small camera movement;
- a reveal that fades if the player turns away.

9.2.4 Held-item transformation

The reliable first implementation is a cinematic transition after two proofs:

1. the player is verified at the pickup location;
2. the expected item appears in the held-item region for several frames.

The presentation may transform an ordinary mermaid gem into a custom relic through video, 2.5D layers, shaders, particles, or a Three.js model.

A future live version MAY estimate the held-item region, segment or detect the item, reproduce its idle sway, and render a custom object with motion derived from optical flow. Perfect hand reconstruction is not required for the baseline.

9.3 Presentation duration and pacing

Cinematic timing is a story decision. The system MUST support deliberate, elaborate transitions rather than forcing short interface-animation conventions. A five-to-fifteen-second sequence is acceptable when it contains meaningful motion and progression. A ten-second static fade is not meaningful motion; it is a hostage situation.

Chapter 10

Vision Waypoints as First-Class Studio Assets

A Vision Waypoint is not code embedded in a Tall Tale. It is a reusable asset with its own authoring data, tests, reliability status, versions, and runtime package.

The waypoint library SHOULD be a primary Studio section alongside Tall Tales, Assets, Characters, Locations, and Publishing.

10.1 Required waypoint metadata

Every waypoint MUST store:

- unique identifier;
- human-readable name;
- optional description and author notes;
- island, region, or logical location tags;
- waypoint type;
- thumbnail;
- owner and sharing scope;
- creation and modification timestamps;
- active draft version;
- published versions;
- reference-recording inventory;
- hard-negative inventory;
- accepted pose definition;
- visual masks and required landmark regions;
- verification profile;
- reliability grade;
- supported runtime modes;
- test history;
- Tall Tales and story blocks using each version;
- known limitations;
- game-build compatibility notes.

10.2 Reuse rules

A waypoint MAY be attached to multiple Tall Tales. The recognition package remains the same while each story configures its own:

- player prompt;
- Captain notification;
- objective-completion action;
- presentation sequence;
- journal unlock;
- next stage;
- fallback behavior.

Waypoint Types

11.1 Exact Landmark

Use for a specific fixed feature such as a rock, statue, doorway, shrine, painting, shipwreck section, or architectural detail.

Typical requirements:

- strong local geometry;
- camera pose within a constrained region;
- target visibility;
- contextual landmark evidence;
- optional planar surface confirmation.

11.2 Area Arrival

Use when the player needs to reach a broader area such as a northern beach, summit, cave chamber, or shrine perimeter.

Typical requirements:

- wider accepted pose volume;
- multiple contextual landmarks;
- sequence or panorama-style scan;
- no reliance on generic sand, water, or sky alone.

11.3 Viewpoint

Use when the camera must be positioned and oriented so that landmarks align or a particular scene is visible.

Typical requirements:

- narrow position region;
- strict camera-facing constraint;
- alignment or silhouette rule;
- minimum overlap among required landmark regions.

11.4 Object Inspection

Use when a particular object must be large and clear enough in the frame.

Typical requirements:

- object-level feature or detector evidence;
- expected scale from estimated distance;
- minimum visible fraction;
- correct background or location context.

11.5 Item Pickup

Use when the player must reach the correct location and then visibly hold an expected item.

Typical requirements:

- successful location verification;
- item appearance in a configured screen region;
- temporal persistence;
- size, pose, or color consistency;
- optional sequence order.

11.6 Sequence Waypoint

Use for a chain such as inspecting three statues, following a path, or finding symbols in order.

Typical requirements:

- child waypoint list;
- ordering rules;
- timeout and reset behavior;
- partial-progress persistence;
- duplicate-event prevention.

11.7 Custom composite

Advanced creators MAY combine existing proof modules. Custom composites MUST still use mandatory gates and publish through the same reliability process.

Chapter 12

Studio Information Architecture

The Studio SHOULD include:

```
Studio
├── Tall Tales
├── Assets
├── Characters
├── Locations
├── Vision Waypoints
├── Test Sessions
└── Publishing
```

12.1 Vision Waypoint library card

Each card SHOULD display:

- thumbnail;
- name;
- type;
- reliability grade;
- latest published version;
- last tested date;
- number of Tall Tales using it;
- status such as Draft, Shadow, Captain-confirmed, Automatic, or Deprecated.

Available actions SHOULD include:

- Open;
- Test;
- Duplicate;
- Create new version;
- Compare versions;
- Export;
- Share;
- Archive;
- View usage;
- Deprecate.

12.2 Story editor integration

A creator drags a `Vision Verification` block into the story graph. The block MUST support:

- `Create New Waypoint`;
- `Choose Existing Waypoint`;
- version selection;
- runtime mode;
- scan prompt and duration preset;
- success event chain;
- insufficient-evidence response;
- not-at-target response;
- Captain fallback;
- offline fallback;
- test launch.

The story graph may then connect:

```
Vision Waypoint: Message Rock
    -> Mark Objective Complete
    -> Fade Game Capture into Scene
    -> Play Rock Carving Reveal
    -> Show Captain Message
    -> Unlock Journal Entry 7
    -> Advance to Mermaid Gem Search
```

Chapter 13

Vision Waypoint Creation Wizard

13.1 Step 1: Define what must be proven

The Studio asks, “What must the player prove?”

Options:

- reached a location;
- found a particular landmark;
- looked from the correct viewpoint;
- found and inspected an object;
- found and held an item;
- completed a sequence;
- advanced custom proof.

The Studio separately asks, “What should happen when verified?” This configures the story connection, not the waypoint package.

13.2 Step 2: Pair the Capture Companion

The Studio MUST support a low-friction local pairing flow through one or more of:

- automatic localhost discovery;
- a short pairing code;
- a QR code;
- an “Open in Companion” deep link.

The connection panel SHOULD show:

- selected window;
- capture status;
- resolution;
- available frame rate;
- hardware accelerator;
- local or cloud-assisted build mode;
- privacy status;
- estimated storage requirement;
- Companion version compatibility.

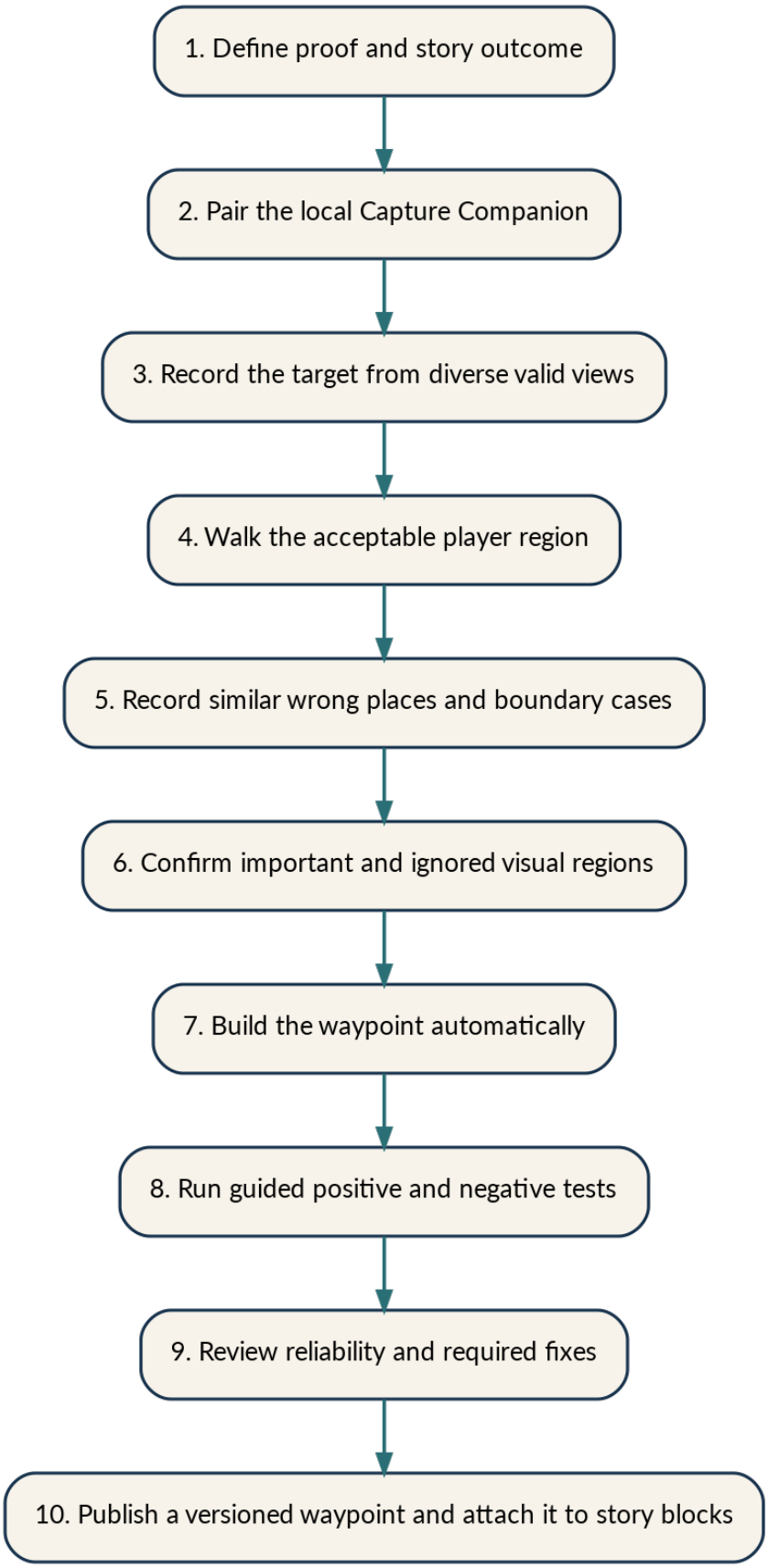


Figure 13.1: Creator waypoint authoring flow

13.3 Step 3: Record the correct location

The creator begins a guided target recording. The Companion and Studio provide live instructions such as:

- keep the target visible;
- move slowly around the landmark;
- capture close, medium, and farther distances;
- record more of the left or right side;
- look slightly up or down;
- move farther back so contextual landmarks are visible;
- pause because the current view is blurry;
- skip a view because it is too similar to existing footage.

13.3.1 Coverage visualization

The Studio SHOULD show:

- angular coverage around the target;
- distance-band coverage;
- camera-pitch coverage;
- visual-diversity coverage;
- lighting-condition coverage when available;
- reconstruction health;
- missing-view recommendations.

The creator sees “Right side incomplete,” not “insufficient baseline triangulation.”

13.4 Step 4: Define the acceptable player region

13.4.1 Guided walk mode

The creator walks through positions where verification should be accepted and faces the target from several points. The system estimates the region from localized camera poses.

13.4.2 Visual editor mode

The Studio shows a simplified 3D or top-down representation. The creator may:

- expand or shrink the region;
- exclude areas;
- set minimum and maximum distance;
- require the camera to remain in front of the target;
- set a facing-angle limit;
- require visibility of named landmark regions;
- create several allowed subregions;
- create a forbidden region;
- designate a preferred cinematic viewpoint.

Plain-language controls MUST be primary. Engineering values MAY appear in Advanced Diagnostics.

13.5 Step 5: Record confusing locations

For Story-Critical and Strict waypoints, hard-negative capture MUST be required before automatic publishing.

The creator records:

- similar objects nearby;
- the opposite side of the target;
- the correct island but wrong landmark;
- just outside the accepted region;
- a correct object viewed from a forbidden side;
- similar distant locations;
- reused props or structures;
- likely false-positive scenes suggested by the Build Engine.

The Studio SHOULD turn this into a guided task: “Record at least two places nearby that look somewhat similar.”

13.6 Step 6: Confirm important and ignored regions

The Studio automatically suggests stable visual regions. The creator can paint or select:

- Important;
- Required context;
- Ignore;
- Dynamic;
- Optional.

Typical ignored content includes:

- water;
- sky and clouds;
- fog;
- grass and leaves;
- players and skeletons;
- ships;
- held items when not part of the checkpoint;
- HUD;
- fire and particles.

Typical important content includes:

- rock cracks;
- cliff edges;
- beams and doorways;
- statue geometry;
- tree trunks;
- fixed prop arrangements;
- the relative geometry between several landmarks.

13.7 Step 7: Automatic build

The user-facing build stages SHOULD be:

1. preparing recorded footage;
2. removing blurry and repeated frames;
3. mapping the landmark;
4. learning its strongest visual details;
5. checking similar locations;
6. building the player package;
7. testing reliability.

The technical stages are specified in Chapter 16.

13.8 Step 8: Guided reliability testing

The Studio schedules positive and negative tests appropriate to the waypoint type.

Positive examples may include:

- standard approach;
- left and right approaches;
- close and far distances;
- difficult lighting;
- alternate FOV;
- a second gameplay session;
- a second player or machine.

Negative examples may include:

- the nearest similar location;
- just outside the accepted region;
- correct area while facing away;
- correct island but wrong target;
- similar location elsewhere;
- obstructed or low-quality scans.

13.9 Step 9: Reliability review and remediation

The Studio MUST explain the problem and prescribe the next action.

Examples:

- “The western rock scored too similarly. Record the target with the curved tree and upper ridge visible.”
- “Close-range valid views are failing. Add close-range target coverage.”
- “Nighttime views contain too few stable details. Record the location at night with the landmark fully visible.”
- “The accepted region is too broad. Reduce the region or add required contextual landmarks.”

13.10 Step 10: Publish and attach

The Studio publishes an immutable waypoint version, records its reliability certificate, and allows it to be attached to story blocks. Unsafe waypoints MUST NOT be published for automatic progression.

Chapter 14

Capture Companion Requirements

14.1 Window capture

The Companion SHOULD use Windows Graphics Capture or an equivalent supported Windows API that captures a user-selected window without modifying the source application. Microsoft documents `Windows.Graphics.Capture` as a secure mechanism for capturing displays and application windows [R1].

The Companion MUST:

- require the user to choose or approve the Sea of Thieves window;
- visibly indicate when capture is active;
- stop when the window closes or permission is revoked;
- avoid capturing unrelated displays by default;
- release frame resources promptly;
- handle window resizing and resolution changes;
- detect menus, loading screens, minimization, and unusable frames;
- expose a pause control and emergency stop hotkey.

14.2 Authoring recording mode

The Companion MUST support named recording sessions for:

- target orbit;
- acceptable-region walk;
- boundary capture;
- nearby hard negative;
- distant hard negative;
- environmental variation;
- live positive test;
- live negative test;
- diagnostic retry.

Each session records metadata including resolution, FOV if known, graphics settings if supplied, time, waypoint draft, creator, and session purpose.

14.3 Runtime scan mode

The default interaction is **press and hold to inspect**.

Initial engineering targets:

Parameter	Initial target
Scan duration	4 to 7 seconds; default 5 seconds
Capture sampling	8 to 12 frames per second
Raw frames	approximately 40 to 60
Retained frames	approximately 8 to 15 high-value frames
Expected motion	slow horizontal or slight multi-axis sweep
Retry cooldown	approximately 2 seconds
Runtime output	VERIFIED, INSUFFICIENT_VISUAL_EVIDENCE, or NOT_AT_TARGET

These are calibration starting points, not universal constants.

14.4 Local communication

The Companion and web surfaces SHOULD communicate through a local authenticated WebSocket or equivalent bidirectional channel.

Requirements:

- localhost binding by default;
- short-lived pairing token;
- origin validation;
- per-session nonce;
- signed or authenticated result messages;
- no unauthenticated remote control;
- explicit capability negotiation;
- version compatibility check;
- heartbeat and clean reconnect;
- attempt IDs for idempotency.

14.5 Hardware detection

The Companion SHOULD select the runtime path automatically:

- NVIDIA GPU: CUDA or TensorRT-capable path where supported;
- Windows ML accelerator: WinML or an appropriate ONNX execution provider;
- CPU: reduced-rate fallback;
- insufficient local capability: cloud-assisted build option for authoring, while preserving a lightweight runtime where possible.

ONNX Runtime supports hardware-specific execution providers and cross-platform deployment [R10]. Current ONNX Runtime guidance recommends WinML for new Windows projects where appropriate and supports CUDA for NVIDIA acceleration [R11].

Chapter 15

Data Collection Methodology

15.1 Target orbit

The creator records overlapping views while moving around the target. The capture SHOULD include:

- direct front;
- front-left and front-right;
- additional side views where valid;
- close, medium, and farther distance bands;
- slight upward and downward pitch;
- natural standing heights;
- slow, stable motion;
- enough background context to distinguish similar targets.

15.2 Acceptable-area coverage

The creator records all areas from which a normal player should succeed. The resulting pose set becomes the basis for the accepted camera volume.

15.3 Boundary capture

The creator records the edge of the accepted region and positions just outside it. This is necessary to teach the system where success must stop.

15.4 Environmental variation

Where practical, a mature waypoint SHOULD include:

- day;
- night;
- sunset or low-angle light;
- overcast;
- storm or fog;
- lantern illumination;
- multiple FOV values;

- multiple resolutions and aspect ratios;
- representative quality settings;
- typical held objects and HUD states.

Not every waypoint requires every condition before pilot use, but missing conditions MUST be recorded as known limitations.

15.5 Hard-negative mining

Initial hard negatives are creator-recorded. Later, high-scoring rejected scans and human-confirmed false results SHOULD be offered as new training or test evidence.

The system MUST distinguish:

- target versus obvious wrong scenes;
- target versus nearby similar scenes;
- target versus the same object from a forbidden side;
- accepted pose versus boundary pose;
- correct location versus correct island;
- correct object versus reused prop.

15.6 Dataset partitioning

Waypoint evidence MUST be divided into:

- build/reference set;
- validation set used to select thresholds;
- locked test set used only after tuning;
- field evidence collected after release.

The same extracted frames MUST NOT be duplicated across partitions in a way that creates leakage.

Chapter 16

Vision Build Engine

The Build Engine transforms recordings into a certified package.

16.1 Processing pipeline

1. ingest recording sessions and metadata;
2. decode frames at a controlled sampling rate;
3. detect and remove menus, loading screens, and invalid captures;
4. score blur, exposure, obstruction, and scenery content;
5. remove near-duplicates;
6. cluster by viewpoint and visual diversity;
7. extract global place descriptors;
8. extract local features;
9. match target frames and build connectivity;
10. perform Structure-from-Motion reconstruction;
11. estimate reference camera poses;
12. map stable regions into reference views and 3D landmarks where possible;
13. ingest negative recordings and generate negative descriptors and features;
14. estimate accepted pose volume from creator walks and boundaries;
15. choose a checkpoint-specific verification profile;
16. select runtime reference subsets that balance coverage and package size;
17. calibrate mandatory thresholds;
18. run offline positive and negative test suites;
19. generate a reliability report and recommended fixes;
20. build and sign the immutable runtime package.

16.2 Candidate technical foundations

The governing behavior does not require one permanent library. The initial candidate stack is:

- HLoc-style hierarchical localization for image retrieval plus feature matching and six-degree-of-freedom localization [R2];
- COLMAP or PyCOLMAP for Structure-from-Motion and reference map creation [R3, R4];
- DINOv2-derived features as a general visual backbone [R5];

- SALAD and/or AnyLoc for coarse visual place recognition [R6, R7];
- SuperPoint plus LightGlue for sparse local feature matching [R8];
- EfficientLoFTR or LoFTR as a secondary semi-dense matcher or disagreement resolver [R9];
- OpenCV for image quality, homography, robust geometry, optical flow, and pose utilities;
- ONNX Runtime or WinML for optimized deployment [R10, R11].

Licenses, model redistribution terms, GPU requirements, package size, and production support MUST be reviewed before final adoption.

Chapter 17

Visual Checkpoint Package Structure

A draft authoring workspace MAY resemble:

```
visual-checkpoints/
└── sailors-bounty-message-rock/
    ├── checkpoint.json
    ├── positive-reference-frames/
    ├── borderline-positive-frames/
    ├── hard-negative-frames/
    ├── reference-embeddings/
    ├── local-features/
    ├── sparse-3d-map/
    ├── stable-region-masks/
    ├── accepted-pose-volume/
    ├── test-dataset/
    └── calibration-report.json
```

The published runtime package MUST be compact and MUST exclude unnecessary raw footage.

17.1 Positive references

Positive references MUST represent the accepted pose volume rather than one magic standing point.

17.2 Borderline positives

Borderline positives SHOULD include valid but difficult views. They are used to prevent excessive false negatives at legitimate boundaries.

17.3 Hard negatives

Hard negatives MUST include the most similar wrong scenes discovered during authoring and testing.

17.4 Stable-region masks

Stable masks identify evidence expected to remain fixed. Required regions MAY be used as mandatory sub-gates.

17.5 Sparse 3D map

The map stores:

- 3D landmark points;
- reference observations;
- reference camera poses;
- feature descriptors or links to them;
- optional semantic or region labels;
- map health and reconstruction quality.

COLMAP is a general-purpose Structure-from-Motion and Multi-View Stereo system, and PyCOLMAP exposes reusable functionality for Python integrations [R3, R4].

Chapter 18

Runtime Verification Protocol

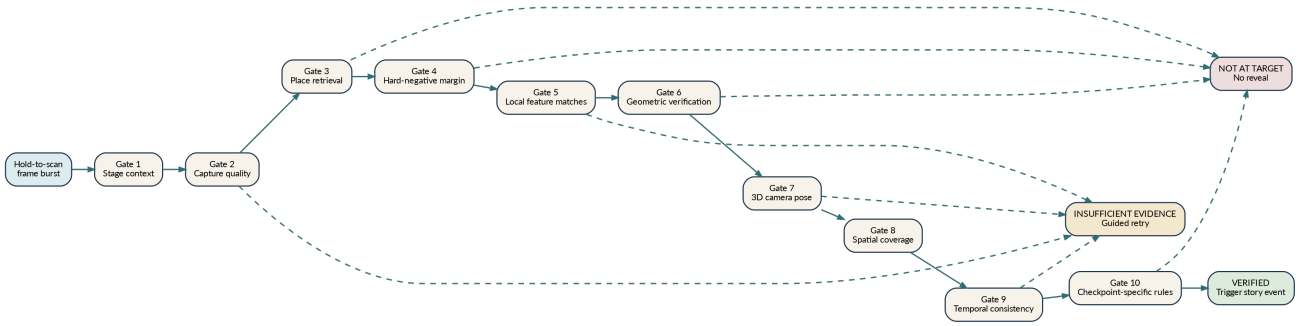


Figure 18.1: Mandatory runtime verification gates

18.1 No weighted-score override

The system MUST NOT use a formula in which one strong score can compensate for failed geometry or a dangerous negative match.

The governing logic is conceptually:

```
verified = (
    story_context_passed
    and capture_quality_passed
    and target_retrieval_passed
    and hard_negative_margin_passed
    and local_feature_geometry_passed
    and camera_pose_passed
    and spatial_coverage_passed
    and temporal_consistency_passed
    and checkpoint_specific_rules_passed
    and not ambiguity_veto
)
```

An internal confidence estimate MAY summarize evidence for diagnostics, but it MUST NOT bypass mandatory gates.

Gate 1: Story and Checkpoint Context

The runtime MUST load only:

- the active waypoint version;
- its configured hard negatives;
- shared negative packs explicitly linked to it;
- required item or object models;
- its player guidance and thresholds.

The system MUST reject stale, mismatched, or unauthorized waypoint versions.

Story context narrows recognition but does not count as visual proof.

Gate 2: Capture Quality

Each raw frame SHOULD be scored for:

- motion blur;
- overexposure;
- underexposure;
- percentage of sky or water;
- percentage of stable scenery;
- obstruction by menus, UI panels, players, weapons, or effects;
- frame duplication;
- capture freshness;
- resolution and scaling validity;
- minimum scene texture;
- excessive camera movement.

Frames that fail are discarded. If too few usable frames remain, the result MUST be `INSUFFICIENT_VISUAL_EVIDENCE` rather than a guess.

Example player guidance:

- “The compass could not settle. Move more slowly.”
- “Step back and keep more of the landmark in view.”
- “Clear the view and inspect again.”

Chapter 21

Gate 3: Coarse Visual Place Retrieval

For each retained frame, the system generates a global descriptor and retrieves likely target references.

Candidate models include SALAD, which builds DINOv2-based descriptors for visual place recognition [R6], and AnyLoc, which uses general pretrained features for place recognition across diverse environments and viewpoints without task-specific retraining [R7]. DINOv2 provides general-purpose visual features that can often be used without immediate fine-tuning [R5].

21.1 Multi-scale and multi-crop retrieval

The system SHOULD evaluate:

- full frame;
- center crop;
- target-region crop when predicted;
- left and right context crops;
- upper terrain or architecture crop;
- several image scales.

This protects against the target occupying only part of the screen and against irrelevant sky, water, or HUD dominating the whole-image descriptor.

21.2 Retrieval result

Gate 3 outputs:

- top target references;
- target similarity distribution;
- top negative references;
- target-negative separation;
- reference viewpoint diversity;
- ambiguity flags.

Retrieval selects candidates. It MUST NOT independently verify the waypoint.

Chapter 22

Gate 4: Hard-Negative Margin and Veto

The verifier MUST compare the target against known confusers.

Minimum logic:

```
target_similarity >= target_minimum
AND
target_similarity - best_negative_similarity >= separation_margin
```

If the target and a negative are close, the attempt MUST be rejected or marked insufficient, even if the raw target score appears high.

A hard-negative veto MAY also use:

- local feature geometry against negative references;
- successful localization inside a negative 3D map;
- reused-prop evidence;
- a correct object with incorrect surrounding context;
- a pose outside the accepted target region but inside a known boundary-negative region.

Chapter 23

Gate 5: Local Feature Matching

The primary candidate is SuperPoint plus LightGlue. LightGlue matches sparse local features across image pairs and adapts computation to pair difficulty [R8].

A secondary matcher SHOULD be available for difficult scenes or disagreement analysis. EfficientLoFTR and LoFTR are candidate detector-free or semi-dense matching systems [R9].

23.1 Multi-matcher policy

The secondary matcher is not required to return identical correspondences. It SHOULD:

- support the primary geometric conclusion;
- provide evidence in low-texture regions;
- or at minimum not strongly contradict the primary result.

Strong disagreement triggers an ambiguity veto or guided retry.

Chapter 24

Gate 6: Geometric Verification

Feature matches MUST form a physically plausible transformation.

24.1 Planar or approximately planar targets

For rock faces, walls, tablets, and paintings, the system MAY estimate a homography with robust outlier rejection.

Required checks SHOULD include:

- minimum raw matches;
- minimum inliers;
- minimum inlier ratio;
- maximum reprojection error;
- plausible scale change;
- plausible perspective distortion;
- adequate spatial coverage;
- no degenerate transformation.

OpenCV provides feature matching and homography tools suitable for locating a known planar surface within a larger image [R12].

24.2 Three-dimensional scenes

For beaches, ruins, caves, and multi-depth scenes, the system MUST prefer camera-pose localization against the sparse 3D map rather than a single homography.

Chapter 25

Gate 7: 3D Camera-Pose Localization

The system associates query features with known 3D landmarks and estimates camera position and orientation.

The output SHOULD include:

- position in local map coordinates;
- yaw, pitch, and roll;
- number of 2D-to-3D correspondences;
- inlier count and ratio;
- reprojection error;
- pose covariance or uncertainty estimate;
- localized reference neighborhood;
- distance and bearing to the target;
- whether the pose lies within the accepted volume.

HLoc is an example of a modular system that combines retrieval, matching, Structure-from-Motion, and six-degree-of-freedom localization [R2].

25.1 Viewpoint tolerance

This gate is the primary solution to the “slightly different from the training image” problem. The player does not need to reproduce an exact reference screenshot. The system determines whether a new view is geometrically consistent with the reconstructed scene and whether the estimated camera lies in an approved region.

Chapter 26

Gate 8: Spatial Distribution of Evidence

Raw match count is insufficient. Matches clustered in one tiny repeated texture are weak evidence.

The verifier SHOULD divide the image or target mask into cells and require evidence across multiple areas.

Examples:

- cave: left wall, right wall, central opening, outside context;
- rock: upper crack, lower edge, adjacent ground, nearby tree or ridge;
- ruin: two structural edges plus one background landmark;
- viewpoint: multiple aligned landmarks at distinct depths.

Required-region masks MAY make particular cells mandatory.

Chapter 27

Gate 9: Temporal and Sequence Consistency

Each retained frame should produce a retrieval and localization result. Valid scans MUST form a plausible camera trajectory.

Expected behavior for a stationary player turning slowly:

- camera position remains nearly constant;
- yaw changes smoothly;
- target identity remains stable;
- reprojection quality remains within limits;
- localized frames overlap coherently.

Implausible behavior includes:

- position jumping across the map;
- alternating between target and negative maps;
- impossible vertical movement;
- random orientation changes inconsistent with optical flow;
- one isolated strong frame surrounded by failures.

The verifier SHOULD require a configured fraction of frames to pass and SHOULD reject isolated peaks.

Chapter 28

Gate 10: Checkpoint-Specific Evidence

Different waypoint types require specialized proof.

28.1 Exact rock or wall

Require:

- valid place retrieval;
- accepted camera pose;
- target-surface homography or local geometry;
- minimum target screen area;
- contextual landmark evidence.

28.2 Cave entrance

Require:

- accepted approach region;
- opening silhouette or depth transition;
- both side walls;
- interior darkness pattern where stable;
- outside context.

28.3 Small statue or object

Require:

- object-level match;
- expected scale for estimated distance;
- minimum visible fraction;
- correct surrounding location.

28.4 Generic beach

Require:

- wider scan;
- several fixed landmarks;
- shoreline or cliff geometry;
- vegetation arrangement only as supporting evidence;
- no acceptance from sand, water, or sky alone.

28.5 Item pickup

Require an ordered combination:

1. location verified;
2. item appears in the configured held-item region;
3. item persists for several frames;
4. optional item-specific appearance rules pass.

The item alone MUST NOT unlock the event because Sea of Thieves reuses loot objects with impressive enthusiasm.

Chapter 29

Accepted Camera Volumes

A waypoint MUST define where the player may stand and, when necessary, where they may face.

29.1 Supported volume forms

The Studio SHOULD support:

- sphere;
- cylinder;
- axis-aligned or oriented box;
- polygonal prism;
- distance band around target;
- union of multiple approved regions;
- exclusion volumes;
- learned hull from captured valid poses.

29.2 Orientation constraints

A waypoint MAY require:

- camera forward ray intersects target bounds;
- yaw within a configured angle;
- pitch within a configured range;
- target center within a screen zone;
- minimum target visibility;
- multiple required landmarks visible simultaneously.

29.3 Example

```
Accepted position: 1.5 to 7 meters from the rock
Allowed side: front and front-left only
Maximum facing error: 55 degrees
Required target visibility: at least 45 percent
Required context: curved tree and upper ridge present
Forbidden: behind the rock or inside the waterline boundary
```

Chapter 30

Verification Outcomes and Player Guidance

30.1 VERIFIED

All mandatory gates pass. The Companion sends a signed result and the story engine processes the idempotent verification event.

Player-facing text may be:

- “The compass locks into place.”
- “Something answers from beneath the stone.”
- “The relic stirs.”

30.2 INSUFFICIENT_VISUAL_EVIDENCE

The view may be correct, but the system lacks reliable proof.

Guidance SHOULD be specific without revealing the answer:

- move more slowly;
- step back;
- keep more of the landmark visible;
- inspect farther left or right;
- wait for the view to clear;
- face the object more directly.

30.3 NOT_AT_TARGET

The evidence supports a different place or condition. Player-facing text SHOULD remain story-appropriate and nontechnical:

- “The compass remains silent.”
- “Nothing answers here.”

30.4 Failure-reason privacy

The Player MUST NOT receive exact coordinates, negative-match names, or spoiler-producing diagnostics. The Captain and Creator may see detailed failure reasons

according to permissions.

Chapter 31

Verification Profiles

31.1 Balanced

Designed for noncritical progression with broad viewpoint acceptance and guided retry.

31.2 Strict

Requires stronger separation, more geometric evidence, and tighter pose constraints.

31.3 Story-Critical

For major reveals. SHOULD require:

- more usable frames;
- larger target-negative margin;
- successful local geometry;
- accepted 3D pose;
- required contextual regions;
- stronger temporal consensus;
- no unresolved matcher disagreement;
- successful negative test suite;
- higher publication standard.

31.4 Custom

Advanced configuration. The Studio MUST display warnings when custom settings reduce safety below a certified profile.

Chapter 32

Reliability Grades and Publishing Gates

32.1 Excellent

- no observed false accepts in the required negative suite;
- strong margin from all known confusers;
- high valid-view first-scan success;
- near-complete success after one guided retry;
- stable performance across supported conditions;
- suitable for automatic progression.

32.2 Good

- no observed false accepts;
- some difficult valid viewpoints;
- suitable for automatic use with guided retry or Captain-confirmed mode, depending on story criticality.

32.3 Needs Improvement

- target and negative scores too close;
- incomplete viewpoint coverage;
- unacceptable false-negative pockets;
- insufficient condition coverage;
- automatic progression blocked.

32.4 Unsafe

- any confirmed wrong location passes;
- serious data leakage or test flaw;
- package integrity failure;
- automatic and Captain-confirmed progression blocked until rebuilt.

Chapter 33

Calibration and Threshold Selection

Thresholds MUST be per waypoint or per well-validated profile, not universal constants.

Candidate calibrated values include:

- minimum target retrieval score;
- minimum target-negative margin;
- minimum local matches;
- minimum inlier count;
- minimum inlier ratio;
- maximum reprojection error;
- minimum spatial coverage;
- minimum localized frames;
- minimum temporal consensus;
- maximum pose uncertainty;
- allowed distance and facing angle;
- minimum target visibility.

Illustrative placeholder only:

```
{
  "retrievalMinimum": 0.84,
  "negativeMarginMinimum": 0.07,
  "minimumPoseInliers": 48,
  "minimumInlierRatio": 0.62,
  "maximumReprojectionErrorPx": 2.8,
  "minimumGridCoverage": 0.55,
  "minimumLocalizedFrames": 5,
  "minimumSequenceAgreement": 0.80
}
```

These numbers MUST NOT be copied into production without empirical calibration.

Chapter 34

Test Strategy

34.1 Positive tests

A waypoint test suite SHOULD include:

- standard valid pose;
- left and right valid poses;
- close and far valid poses;
- valid boundary poses;
- alternate FOV;
- alternate resolution;
- day and night when supported;
- environmental obstruction;
- second gameplay session;
- second machine or player when possible.

34.2 Negative tests

A waypoint test suite MUST emphasize:

- nearest similar target;
- correct island, wrong landmark;
- correct landmark, forbidden side;
- just outside accepted region;
- correct area while facing away;
- similar location elsewhere;
- low-quality or obstructed scans;
- stale or wrong waypoint package;
- replayed frames where replay protection applies.

34.3 Offline playback

Recorded scans SHOULD be replayable through the full verifier. This allows thousands of negative attempts without manual gameplay.

34.4 Hard-negative mining loop

1. run all negative footage;
2. rank the highest-scoring rejected attempts;
3. inspect borderline cases;
4. add confirmed confusers;
5. recalibrate;
6. rerun the locked test suite;
7. repeat until release criteria pass.

Chapter 35

Statistical Reliability

Observing zero errors in a small test does not prove a zero error rate.

For zero observed false accepts across `N` independent negative attempts, the approximate 95 percent upper confidence bound is:

$$p_{\text{false accept}} \lesssim \frac{3}{N}$$

Examples:

Negative attempts	Zero observed false accepts implies approximate upper bound
300	about 1 percent
3,000	about 0.1 percent
30,000	about 0.01 percent

The first personal prototype does not need 30,000 manual scans. Offline playback, shared negative packs, and automated regression testing SHOULD increase coverage.

A release report SHOULD state:

- number of positive attempts;
- first-scan success rate;
- success after one guided retry;
- number of negative attempts;
- observed false accepts;
- confidence bound or equivalent statistical context;
- supported conditions;
- known exclusions.

Chapter 36

Shadow Mode and Promotion Lifecycle

Figure 36.1: Waypoint maturity lifecycle

36.1 Draft

Authoring data is incomplete or unbuilt.

36.2 Built

Package builds and passes structural checks but lacks sufficient field evidence.

36.3 Shadow

Verifier evaluates real attempts but cannot alter progression. Human truth labels are collected.

36.4 Captain-confirmed

Verifier proposes success; Captain approves. This mode gathers live evidence without trusting automatic advancement completely.

36.5 Automatic

Waypoint meets release criteria for its profile and may progress the story directly.

36.6 Demotion

Any confirmed false positive, game-environment change, package corruption, or major model regression SHOULD automatically demote or disable automatic mode until review.

Chapter 37

Captain Controls and Diagnostics

The Captain interface SHOULD provide:

- active waypoint and version;
- current Player and story stage;
- attempt timestamp and status;
- summarized gate results;
- selected query frames;
- matched target references;
- nearest hard-negative reference;
- target-negative margin;
- camera-pose visualization;
- accepted-volume result;
- temporal consistency result;
- checkpoint-specific evidence;
- recommended retry guidance;
- Approve;
- Reject;
- Trigger Presentation Manually;
- Mark Attempt as Positive;
- Mark Attempt as Hard Negative;
- Improve Waypoint From Attempt.

The Player MUST not see this evidence.

37.1 Manual override

Manual override is required fault tolerance. It MUST:

- record the Captain identity;
- record reason and timestamp;
- mark whether the vision result was correct, false negative, or unusable;
- prevent duplicate story events;
- optionally add the evidence to a waypoint-improvement queue.

Chapter 38

Player Experience

38.1 Activation

When the stage activates, the Player surface may display:

> The compass grows restless. Hold **Inspect Surroundings** and slowly search the area.

The Companion status MUST clearly show whether vision is ready, unavailable, paused, or disconnected.

38.2 During scan

The Player SHOULD see:

- a deliberate scan animation;
- hold progress;
- gentle motion guidance;
- privacy indicator;
- a cancel option;
- no raw camera or model diagnostics unless intentionally designed as part of the story.

38.3 Result pacing

Verification may take a short moment after release. The interface SHOULD continue the story metaphor rather than show a generic spinner.

38.4 Accessibility

The scan interaction MUST support:

- remappable keyboard or controller input;
- toggle-to-scan alternative for users unable to hold a key;
- text and audio guidance;
- reduced-motion mode for presentation effects;
- high-contrast status indicators;
- captions and transcript support for audio reveals;

- no critical information conveyed by color alone.

Chapter 39

Story Event Contracts

Verification and presentation MUST communicate through events.

39.1 Example success event

```
{
  "event": "vision.waypoint.verified",
  "attemptId": "att_01K...",
  "waypointId": "vp_01KSBMR7",
  "waypointVersion": 3,
  "storyId": "tt_01K...",
  "stageId": "stage_message_rock",
  "playerId": "player_01K...",
  "timestamp": "2026-07-17T23:40:12-04:00",
  "result": "VERIFIED",
  "evidenceSummary": {
    "usableFrames": 10,
    "localizedFrames": 7,
    "profile": "story_critical"
  },
  "signature": "..."
}
```

39.2 Idempotency

The story engine MUST process each attempt or verification event once. Reconnects and retries MUST NOT replay a reveal or advance the story twice.

39.3 Separation of concerns

The verifier MUST NOT directly write journal entries, play videos, or select the next stage. It emits proof. The story engine owns consequences.

Chapter 40

Presentation Asset Integration

The story presentation block SHOULD support:

- image reveal;
- WebM or MP4 cinematic;
- audio cue or narration;
- journal entry;
- Three.js scene;
- 2.5D parallax scene;
- surface-anchored texture;
- held-item transformation;
- chained presentation;
- delayed or conditional presentation;
- Captain-visible preview.

40.1 View-conditioned assets

A presentation may query the verified pose class:

```
viewpoint: front_left
range: medium
pitch: slightly_down
lighting: night
```

The presentation engine then chooses the closest supported asset. It MUST fall back gracefully to a general asset when no exact variant exists.

Chapter 41

Live External AR Requirements

41.1 Surface reveal

A live surface effect MAY use:

- target-surface feature correspondences;
- homography or mesh warp;
- repeated re-localization;
- optical flow between heavy localization frames;
- alpha-masked textures;
- perspective-aware shaders;
- fade-out when confidence drops.

The effect MUST disappear safely when tracking is lost. It MUST NOT drift visibly across unrelated geometry for extended periods.

41.2 Held-object replacement

A future live object effect MAY use:

- expected held-item screen region;
- object detector or segmenter;
- pose or bounding-box smoothing;
- optical-flow-derived sway;
- a custom 3D model;
- hand-occlusion masks where practical;
- controlled cut to a cinematic when live confidence is insufficient.

The baseline SHOULD prefer reliable cinematic substitution over fragile “perfect” real-time replacement.

Chapter 42

Local, Cloud-Assisted, and Shared Processing

42.1 Local-first runtime

Runtime verification MUST be local by default to reduce latency, protect privacy, and avoid service dependency.

42.2 Local build

A capable creator machine MAY build maps and packages locally.

42.3 Cloud-assisted build

The Studio MAY offer cloud assistance when:

- the creator lacks a suitable GPU;
- reconstruction is too slow;
- centralized models or validation are required;
- a shared waypoint pack is being curated.

Cloud-assisted processing MUST clearly disclose what data is uploaded and how long it is retained.

42.4 Derived-data upload

Where feasible, the system MAY upload selected frames or derived descriptors instead of full recordings. This must be evaluated against accuracy, privacy, and reproducibility requirements.

Chapter 43

Privacy, Security, and Game-Safety Boundaries

43.1 Privacy requirements

The Companion MUST:

- capture only the approved game window by default;
- show a visible Vision Active indicator;
- process frames locally by default;
- discard runtime frames immediately after inference unless diagnostics are explicitly enabled;
- retain no continuous recording by default;
- provide pause and stop controls;
- explain when a diagnostic attempt will be saved;
- support deletion of creator recordings and field evidence;
- avoid transmitting raw screenshots to the website unless required for an authorized presentation or diagnostic action.

The website SHOULD receive a compact result rather than continuous imagery.

43.2 Security requirements

- Pairing tokens MUST expire.
- Runtime packages MUST be integrity checked.
- Results MUST include attempt IDs and authentication.
- Package downloads SHOULD be signed or hash-verified.
- The Companion MUST reject untrusted remote commands.
- Diagnostic assets MUST be access controlled.
- Waypoint sharing MUST respect scope and ownership.
- Cloud uploads MUST use encrypted transport and documented retention.

43.3 Sea of Thieves safety boundary

Sea of Thieves support states that third-party tools used to modify the visual experience, including ReShade-style filters, may be bannable [R13]. Therefore, the project MUST

maintain a conservative separation:

- no code injection;
- no game-memory access;
- no game-file modification;
- no input automation;
- no in-game overlay;
- no competitive assistance;
- effects appear only in the separate companion surface;
- screen capture is read-only and user-selected.

Before public distribution, the project SHOULD seek written clarification from Rare or Microsoft regarding the exact external read-only companion behavior. The current governing design minimizes risk but does not claim an official endorsement.

Chapter 44

Performance Budgets

Initial targets SHOULD be measured on representative hardware.

44.1 Runtime scan

- responsive capture start after hold begins;
- no material impact on game frame rate under normal settings;
- analysis completes quickly enough that the story remains fluid;
- GPU and CPU usage stop promptly after the attempt;
- memory is bounded and frame buffers are released;
- package loading is cached and stage-aware.

44.2 Sampling strategy

The verifier SHOULD process selected frames rather than full-rate 60/120/165 Hz video. Initial design uses 8 to 12 sampled frames per second during a five-second scan, then retains only the most useful subset.

44.3 Degraded operation

If hardware cannot sustain the preferred pipeline, the Companion MAY:

- lower image resolution;
- reduce retained frame count within certified limits;
- use a lighter model;
- disable the secondary matcher unless ambiguity requires it;
- increase analysis time slightly;
- require Captain-confirmed mode;
- refuse automatic mode if reliability cannot be preserved.

Performance degradation MUST NOT silently lower verification safety.

Chapter 45

Observability and Diagnostic Evidence

Each attempt SHOULD produce a structured internal record containing:

- attempt and session IDs;
- waypoint and version;
- Companion and model versions;
- hardware path;
- frame-quality summary;
- candidate references;
- target and negative scores;
- matcher and geometry summaries;
- pose estimates and uncertainty;
- temporal consensus;
- checkpoint-specific evidence;
- final result and guidance reason;
- Captain action when applicable;
- privacy and retention flags.

Raw frames MUST be omitted unless the user has enabled diagnostic retention for that attempt.

45.1 Truth labeling

Creators and Captains SHOULD be able to label an attempt:

- correct success;
- correct rejection;
- false negative;
- false positive;
- unusable input;
- uncertain human judgment.

False positives MUST trigger urgent review and potential automatic-mode demotion.

Chapter 46

Versioning, Sharing, and Marketplace Foundations

46.1 Versioning

Every published waypoint version is immutable. A new version is created for:

- additional target coverage;
- new hard negatives;
- changed thresholds;
- new model or feature format;
- changed accepted region;
- updated game environment;
- improved presentation viewpoint mapping.

46.2 Compatibility

Packages MUST declare:

- package schema version;
- minimum Companion version;
- model runtime requirements;
- supported FOV or resolution ranges where known;
- game-build compatibility notes;
- deprecation status.

46.3 Sharing scopes

- Private;
- Crew or collaborators;
- Tall Tale only;
- Community;
- Official or curated.

46.4 Future marketplace

A shared waypoint catalog MAY allow creators to use prepared location packs. A community waypoint SHOULD expose:

- reliability certificate;
- supported conditions;
- user success reports;
- known limitations;
- last verified game version;
- owner and license;
- package version history.

A public catalog is a future capability. The internal asset and version model MUST support it from the start.

Chapter 47

Model Training Strategy

47.1 Do not begin with custom training

The baseline SHOULD use pretrained retrieval and matching systems, per-waypoint references, hard negatives, geometry, and calibration.

Custom training introduces:

- dataset labeling burden;
- training infrastructure;
- model distribution and licensing concerns;
- overfitting risk;
- waypoint maintenance;
- harder explainability.

47.2 When custom training becomes justified

Fine-tuning or metric learning MAY be considered when evidence shows repeatable failure that cannot be solved through:

- better target coverage;
- better hard negatives;
- improved stable masks;
- stronger local geometry;
- better 3D mapping;
- model substitution;
- checkpoint-specific rules.

47.3 Candidate custom objective

A Siamese or triplet-loss model may learn:

- target and valid view: close embedding;
- target and similar wrong rock: separated embedding;
- borderline valid views: accepted but lower-confidence neighborhood.

Any trained model MUST remain one gate within the system, not the sole authority.

Chapter 48

Risk Register and Mitigations

Risk	Consequence	Governing mitigation
False positive	Story reveal occurs early or at wrong place	mandatory gates, hard negatives, geometry, pose, temporal consensus, shadow testing, automatic demotion
False negative	Player frustration and damaged trust	3D viewpoint tolerance, broad valid coverage, guided retry, Captain override, field learning
Similar reused environments	Ambiguous retrieval	current-stage restriction, contextual landmarks, hard-negative maps, spatial coverage
Dynamic weather and lighting	weak matches	diverse capture, robust features, local geometry, supported-condition declarations
Generic texture	accidental feature matches	required distributed evidence and contextual landmarks
Creator under-captures location	brittle package	live coverage guidance and build remediation
Creator skips negatives	dangerous publication	required negative capture for strict profiles and publish blocking
Game update changes scenery	regression	version compatibility, field monitoring, demotion, rebuild workflow
Companion hurts game performance	bad experience	sampled frames, hardware adaptation, bounded buffers, profiling

Risk	Consequence	Governing mitigation
Privacy concern	loss of trust	local-only default, visible indicator, no retention by default
Anti-cheat concern	account risk	external read-only capture, no injection, no overlay, seek clarification before public distribution
Model/license conflict	distribution blocked	dependency inventory, license review, pluggable adapters
Cloud outage	waypoint unavailable	local runtime packages and offline story behavior
Package tampering	false event or crash	signed/hash-verified packages and authenticated events

Chapter 49

Software Architecture Boundaries

The implementation SHOULD isolate replaceable engines behind stable interfaces.

```
apps/
├── studio-web/
├── player-web/
├── captain-web/
└── vision-companion/

services/
├── story-engine/
├── waypoint-catalog/
├── vision-build-orchestrator/
└── optional-cloud-build-worker/

packages/
├── vision-contracts/
├── waypoint-schema/
├── presentation-contracts/
├── companion-protocol/
└── shared-ui/

vision/
├── capture/
├── frame-quality/
├── retrieval/
├── negative-veto/
├── local-matching/
├── geometry/
├── localization/
├── temporal/
├── profiles/
├── calibration/
├── packaging/
└── diagnostics/
```

49.1 Adapter rules

Each model or library MUST be behind an adapter that returns governing contract types. Story code MUST NOT import model-specific libraries directly.

49.2 Deterministic tests

Captured test sequences and expected gate results SHOULD be stored as reproducible fixtures. Model version changes MUST run regression tests before package promotion.

Chapter 50

Codex Implementation Work Packages

The implementation MUST be incremental. Each work package should produce a reviewable, testable vertical slice and update the project documentation.

50.1 Work Package B1: Foundation and contracts

Deliver:

- Vision Waypoint domain model;
- package schema draft;
- Companion pairing protocol;
- three-state result contract;
- attempt IDs and idempotency;
- Studio library shell;
- story `Vision Verification` block shell;
- manual and shadow runtime modes;
- privacy and safety boundary tests.

Definition of done:

- a dummy verifier can return each result;
- the story engine responds correctly;
- no model code is required yet;
- duplicate attempts cannot replay a story event.

50.2 Work Package B2: Capture Companion and guided scan

Deliver:

- window selection;
- Windows capture;
- hold-to-scan;
- frame buffer;
- capture-quality metrics;
- retained-frame selection;
- local WebSocket connection;
- Player scan UI;

- diagnostics without raw frame retention by default.

Definition of done:

- a five-second scan produces a bounded retained frame set;
- menus and blurry scans produce insufficient evidence;
- capture indicator and pause control work;
- game window closure is handled safely.

50.3 Work Package B3: Pilot waypoint authoring

Deliver:

- creation wizard steps 1 through 6;
- target recording;
- acceptable-region walk;
- negative recording;
- coverage guidance;
- stable and ignored region editor;
- recording metadata and storage.

Definition of done:

- one creator can author a complete pilot dataset without touching files or code;
- all sessions are traceable to a waypoint draft;
- the Studio identifies missing target or negative coverage.

50.4 Work Package B4: Coarse retrieval and hard negatives

Deliver:

- global descriptor adapter;
- reference indexing;
- multi-crop retrieval;
- hard-negative comparison;
- target-negative margin;
- offline test runner;
- diagnostic visualizations.

Definition of done:

- target views retrieve appropriate references;
- similar wrong places are explicitly compared;
- close ties do not verify;
- the pipeline remains in shadow mode.

50.5 Work Package B5: Local matching and geometry

Deliver:

- primary feature extractor and matcher;
- optional secondary matcher;
- robust inlier estimation;
- homography profile;
- spatial coverage checks;
- required-region rules.

Definition of done:

- the pilot exact-landmark waypoint rejects a similar wrong rock through geometry;
- clustered matches alone cannot pass;
- diagnostics show inliers and failure reasons.

50.6 Work Package B6: 3D mapping and camera localization

Deliver:

- COLMAP/PyCOLMAP build adapter;
- sparse map storage;
- reference camera poses;
- query-to-3D localization;
- accepted pose volume editor and runtime check;
- pose uncertainty and reprojection metrics.

Definition of done:

- valid viewpoints not identical to references can localize;
- forbidden-side and out-of-region views fail;
- map build failures produce actionable Studio guidance.

50.7 Work Package B7: Multi-frame consensus and profiles

Deliver:

- sequence-level pose smoothing;
- optical-flow or motion consistency;
- isolated-peak rejection;
- Balanced, Strict, and Story-Critical profiles;
- checkpoint-specific rule system.

Definition of done:

- several consistent frames are required;
- impossible camera jumps fail;
- a Story-Critical pilot requires all mandatory gates.

50.8 Work Package B8: Calibration, reliability, and publishing

Deliver:

- positive and negative test manager;
- validation and locked-test partitions;
- threshold calibration;
- reliability grades;
- release report;
- publication blocking;
- immutable package versioning;
- shadow and Captain-confirmed promotion.

Definition of done:

- an unsafe waypoint cannot be published automatically;
- the pilot has a reproducible calibration report;
- package version changes do not alter existing story references.

50.9 Work Package B9: Captain operations and field learning

Deliver:

- live attempt queue;
- evidence summary;
- approve, reject, and manual trigger;
- truth labels;
- improvement queue;
- automatic-mode demotion on confirmed false positive.

Definition of done:

- a Captain can recover a legitimate failed scan without breaking story history;
- attempts can become positive or hard-negative evidence with explicit consent.

50.10 Work Package B10: Production hardening

Deliver:

- package signing or hash validation;
- hardware fallback;
- installer and updater integration;
- performance profiling;
- privacy audit;
- security review;
- anti-cheat boundary documentation;
- regression suite;
- creator help and onboarding.

Chapter 51

Definition of Done for Phase B

Phase B is complete only when:

- a nontechnical creator can author a waypoint using the Studio wizard;
- the system produces a reusable, versioned package;
- a Player can perform a guided scan without seeing technical diagnostics;
- the verifier uses stage context, quality filtering, hard negatives, local geometry, pose, spatial coverage, temporal consensus, and checkpoint-specific rules;
- a valid viewpoint not identical to a reference can pass;
- known similar wrong places fail;
- insufficient evidence produces guided retry rather than arbitrary failure;
- the Captain can override and label attempts;
- shadow and automatic modes are distinct;
- an unsafe waypoint cannot be published for automatic progression;
- runtime capture is local-first and nonretaining by default;
- no prohibited game integration exists;
- automated tests cover idempotency, package compatibility, privacy defaults, and failure handling;
- the pilot waypoint has a documented reliability report.

Chapter 52

Open Decisions Requiring Controlled Resolution

The following are intentionally left as decisions rather than silently assumed facts:

1. final primary visual place-recognition model;
2. final secondary matcher and when it runs;
3. whether runtime models ship in PyTorch, ONNX, WinML, TensorRT, or a mixed stack;
4. local file format for sparse maps and descriptors;
5. exact package signing mechanism;
6. whether the initial Companion is implemented in C#, Rust, C++, Python, or a hybrid;
7. cloud-assisted build hosting and retention policy;
8. first pilot waypoint and its required conditions;
9. minimum negative-test count for each certification profile;
10. Rare/Microsoft clarification before public distribution;
11. marketplace licensing and moderation;
12. whether automatic background listening is introduced in Phase C or deferred.

Each decision SHOULD be captured in an ADR. None may weaken the governing safety and reliability behavior.

Chapter 53

Persistent Domain and Database Model

The implementation MAY use relational tables, document records, or another durable store, but the logical entities and relationships below are governing.

53.1 VisionWaypoint

Represents the stable identity of a waypoint across versions.

Required fields:

- `id`;
- `ownerId`;
- `name`;
- `description`;
- `type`;
- `locationTags`;
- `sharingScope`;
- `archivedAt`;
- `createdAt`;
- `updatedAt`.

A waypoint identity MUST NOT contain mutable runtime model data directly.

53.2 VisionWaypointVersion

Represents one immutable published or mutable draft version.

Required fields:

- `id`;
- `waypointId`;
- semantic or monotonic version number;
- lifecycle status;
- verification profile;
- package schema version;
- draft configuration;
- package artifact reference;
- compatibility metadata;

- certification reference;
- created-by and published-by identities;
- creation and publication timestamps;
- superseded or deprecated status.

Only a draft version may be edited. Publishing creates or seals an immutable version.

53.3 CaptureSession

Represents one creator recording or player scan.

Required fields:

- `id`;
- `waypointVersionId`;
- session purpose;
- creator or player identity;
- Companion version;
- capture resolution;
- frame rate and duration;
- hardware path;
- consent and retention flags;
- start and completion timestamps;
- status and failure reason.

Authoring sessions may retain footage according to creator choices. Runtime sessions default to derived evidence only.

53.4 RecordingAsset

Represents a raw video, extracted frame set, or derived recording artifact.

Required fields:

- asset type;
- storage location;
- content hash;
- source session;
- dataset partition;
- positive, borderline, negative, or unknown truth label;
- creator label;
- automated quality summary;
- deletion status.

53.5 VisualRegion

Represents a polygon, mask, bounding region, or semantic region.

Region categories:

- important;

- required;
- context;
- optional;
- ignored;
- dynamic;
- target surface;
- held-item area.

Regions MUST be versioned with the waypoint draft and transformed consistently across reference views where supported.

53.6 PoseRegion

Represents accepted or forbidden camera-space geometry.

Required fields:

- coordinate-system identifier;
- shape type;
- vertices or parameters;
- accepted or forbidden classification;
- orientation rules;
- target visibility rules;
- authoring source, such as walked poses or manual edit.

53.7 HardNegativeSet

Represents grouped confusers.

Examples:

- nearby gray rocks;
- same island wrong beach;
- target rear side;
- repeated shrine architecture;
- accepted-region boundary;
- community shared confuser pack.

The package MUST preserve which confuser caused the highest risk during certification.

53.8 VisionBuildJob

Represents one build attempt.

Required fields:

- job ID;
- waypoint draft version;
- engine and model versions;
- local or cloud execution;
- processing stage;

- progress;
- logs and structured warnings;
- output artifacts;
- failure code;
- start and finish timestamps.

53.9 CertificationRun

Represents calibration and locked-test evaluation.

Required fields:

- build artifact hash;
- validation partition hash;
- locked-test partition hash;
- thresholds;
- profile;
- positive and negative metrics;
- observed false results;
- reliability grade;
- approved runtime modes;
- reviewer identity;
- report artifact.

53.10 VerificationAttempt

Represents a runtime attempt.

Required fields:

- attempt ID;
- active story and stage;
- waypoint identity and version;
- Player identity;
- capture-session reference;
- result;
- guidance code;
- structured gate summary;
- event-delivery status;
- Captain action;
- human truth label;
- retention policy;
- timestamps.

53.11 StoryWaypointBinding

Connects one immutable waypoint version to one story block.

It stores:

- story and stage IDs;
- waypoint version;
- runtime mode;
- scan interaction settings;
- success action node;
- retry and failure messaging;
- Captain fallback policy;
- offline behavior;
- presentation pose preferences.

Chapter 54

Studio Screen-by-Screen Requirements

54.1 Waypoint library screen

The library MUST support search and filtering by:

- name;
- type;
- location;
- owner;
- reliability;
- lifecycle status;
- sharing scope;
- Tall Tale usage;
- last tested or modified date.

The library SHOULD warn when:

- a published version is used by an active story;
- a newer version is available but untested;
- a game compatibility note is stale;
- a waypoint has been demoted;
- raw authoring assets are missing;
- a package requires a newer Companion.

54.2 Waypoint overview screen

The overview SHOULD include:

- large thumbnail;
- waypoint purpose;
- current draft and published versions;
- reliability grade and allowed modes;
- visual coverage summary;
- negative coverage summary;
- accepted-region preview;
- known limitations;
- linked Tall Tales;
- last field attempts;

- primary actions: Continue Building, Test, Publish Version, Duplicate, and Open Diagnostics.

54.3 Record target screen

The recording screen MUST show:

- live Companion state;
- selected game window;
- recording timer;
- current quality status;
- coverage map;
- next best capture instruction;
- pause, discard, and complete actions;
- an explanation of local storage and retention.

The Studio SHOULD prevent accidental completion when reconstruction coverage is clearly inadequate, but MAY allow “Save as incomplete draft.”

54.4 Accepted-region screen

The default interface SHOULD combine:

- a top-down or 3D point-cloud preview;
- recorded valid camera markers;
- boundary markers;
- a simple distance and facing summary;
- drag handles or shape editing;
- named required landmarks;
- validation warnings.

A creator SHOULD be able to click a recorded camera pose and view its screenshot.

54.5 Similar wrong places screen

Each negative card SHOULD show:

- thumbnail;
- label;
- source location notes;
- highest similarity to the target;
- whether local geometry also matches;
- whether it is included in the runtime negative index;
- recommended additional capture.

The screen MUST visually emphasize the strongest confuser.

54.6 Region-marking screen

The region editor SHOULD provide:

- brush and polygon tools;
- adjustable opacity;
- Important, Required, Context, Ignore, and Dynamic modes;
- undo and redo;
- reference-frame navigation;
- automatic propagation suggestion;
- warning when required regions are not visible from portions of the accepted pose volume.

54.7 Build screen

The Build screen SHOULD stream high-level progress while retaining expandable technical logs.

Build controls:

- Start Build;
- Cancel;
- Retry failed stage;
- Run locally;
- Use cloud assistance;
- Download diagnostic bundle;
- Compare with previous build.

54.8 Test screen

The Test screen MUST distinguish:

- required tests;
- optional robustness tests;
- completed tests;
- passed, failed, and inconclusive attempts;
- tests invalidated by a new build.

The Studio SHOULD launch the correct Companion recording mode automatically for each test case.

54.9 Certification screen

The Certification screen SHOULD show:

- grade;
- approved runtime modes;
- positive success rates;
- negative-attempt count;

- false accept and false reject counts;
- strongest confuser;
- least reliable valid pose class;
- unsupported conditions;
- exact package and test hashes;
- remediation actions;
- Publish or Return to Draft.

Chapter 55

Runtime Attempt State Machine

A verification attempt MUST use a well-defined state machine.

```
IDLE
  -> ARMED
  -> CAPTURING
  -> CURATING_FRAMES
  -> RETRIEVING
  -> MATCHING
  -> LOCALIZING
  -> EVALUATING_SEQUENCE
  -> EVALUATING_SPECIAL_RULES
  -> VERIFIED | INSUFFICIENT | NOT_AT_TARGET | ERROR | CANCELLED
  -> EVENT_DELIVERED or RESULT_DISPLAYED
  -> CLOSED
```

55.1 State rules

- Only one attempt per Player and waypoint SHOULD be active unless multi-player behavior explicitly permits more.
- Releasing the hold control ends capture and starts analysis.
- Cancelling during capture produces no truth label and no story event.
- A Companion disconnect during analysis produces a recoverable error, not a negative result.
- A stale attempt MUST NOT complete after the story has moved to a different stage.
- A verified result MUST be checked against current story state immediately before event delivery.
- Result display and event delivery MUST be idempotent.

Chapter 56

Build Job State Machine and Failure Codes

56.1 Build stages

```
QUEUED
-> INGESTING
-> CURATING
-> EXTRACTING_GLOBAL_FEATURES
-> EXTRACTING_LOCAL_FEATURES
-> MATCHING_REFERENCE_GRAPH
-> RECONSTRUCTING
-> BUILDING_NEGATIVE_INDEX
-> ESTIMATING_POSE_VOLUME
-> CALIBRATING
-> RUNNING_TESTS
-> PACKAGING
-> SIGNING
-> COMPLETE
```

56.2 Recoverable warnings

Examples:

- `LOW_VIEW_DIVERSITY`;
- `MISSING_CLOSE_RANGE`;
- `MISSING_FAR_RANGE`;
- `WEAK_NIGHT_COVERAGE`;
- `TOO_FEW_HARD_NEGATIVES`;
- `REQUIRED_REGION_NOT_VISIBLE_FROM_ALL_VALID_POSES`;
- `HIGH_PACKAGE_SIZE`;
- `SECONDARY_MATCHER_DISAGREEMENT`;
- `LIMITED_FOV_SUPPORT`.

56.3 Blocking failures

Examples:

- `NO_RECONSTRUCTION`;
- `DEGENERATE_CAMERA_GEOMETRY`;
- `TARGET_NEGATIVE_NOT_SEPARABLE`;
- `FALSE_ACCEPT_IN_LOCKED_TEST`;
- `PACKAGE_INTEGRITY_FAILURE`;
- `UNSUPPORTED_MODEL_LICENSE`;
- `NO_VALID_ACCEPTED_POSE`;
- `REQUIRED_ASSET_MISSING`;
- `INCOMPATIBLE_SCHEMA`.

Every failure MUST map to creator-facing remediation. “Build failed” alone is forbidden because it communicates roughly the same useful information as a smoke alarm labeled “something.”

Chapter 57

Companion Protocol Requirements

57.1 Pairing handshake

The handshake SHOULD exchange:

- Studio session ID;
- Companion instance ID;
- short-lived pairing token;
- supported protocol versions;
- installed model bundles;
- available accelerators;
- capture capabilities;
- privacy mode;
- package cache status.

The Studio selects the newest mutually compatible protocol.

57.2 Core message categories

57.2.1 Companion status

```
{
  "type": "companion.status",
  "connected": true,
  "captureState": "ready",
  "selectedWindow": "Sea of Thieves",
  "hardwareProvider": "cuda",
  "privacyMode": "local_no_retention"
}
```

57.2.2 Authoring command

```
{
  "type": "authoring.record.start",
  "sessionId": "cap_01K...",
  "waypointDraftId": "vpv_draft_01K...",
  "purpose": "hard_negative",
```

```
  "label": "western_similar_rock"
}
```

57.2.3 Live guidance

```
{
  "type": "authoring.guidance",
  "code": "CAPTURE_MORE_RIGHT_SIDE",
  "severity": "info",
  "progress": 0.72
}
```

57.2.4 Runtime attempt

```
{
  "type": "runtime.scan.start",
  "attemptId": "att_01K...",
  "waypointId": "vp_01K...",
  "waypointVersion": 3,
  "stageToken": "signed-short-lived-token"
}
```

57.2.5 Result

```
{
  "type": "runtime.scan.result",
  "attemptId": "att_01K...",
  "result": "INSUFFICIENT_VISUAL_EVIDENCE",
  "guidanceCode": "STEP_BACK_FOR_CONTEXT",
  "evidenceDigest": "sha256:...",
  "signature": "..."
}
```

57.3 Protocol error handling

The protocol MUST distinguish:

- user cancellation;
- capture permission loss;
- game window unavailable;
- package missing;
- model unavailable;
- incompatible version;
- insufficient hardware;
- internal inference error;
- stale stage token;
- authentication failure.

None of these should be mislabeled as “not at the target.”

Chapter 58

Default Profile Matrix

The values below describe relative behavior, not final numeric thresholds.

Requirement	Balanced	Strict	Story-Critical
Minimum usable frames	medium	high	highest
Target-negative margin	medium	high	highest
Local geometry	required for exact landmarks	required	required
3D pose	preferred or type-dependent	required for location types	required
Required context regions	optional	usually required	required
Temporal consensus	medium	high	highest
Secondary matcher	on ambiguity	more frequent	required on configured difficult cases
Negative test suite	standard	expanded	expanded plus strongest-confuser replay
Automatic publish	allowed with Good/Excellent	Excellent preferred	Excellent unless explicit Captain-confirmed exception
Captain fallback	enabled	enabled	enabled and logged

Chapter 59

Data Retention Matrix

Data	Default retention	Creator control	Runtime necessity
Raw target recordings	retained for authoring until deleted or archived	yes	no
Raw hard-negative recordings	retained for authoring until deleted or archived	yes	no
Extracted reference frames	retained with draft/build workspace	yes	no after package publication if rebuild assets intentionally removed
Runtime scan frames	discarded after inference	diagnostic opt-in per attempt or session	no
Derived runtime metrics	retained as attempt record	configurable policy	useful for reliability and audit
Captain truth label	retained	project policy	useful for field learning
Published package	retained and immutable	deprecate, not mutate	yes
Certification report	retained with package version	no destructive edit after publication	yes for governance
Cloud build upload	deleted according to disclosed policy	creator chooses cloud use	no after build unless explicitly retained

Chapter 60

Release and Regression Checklist

A waypoint version cannot enter Automatic mode until the following are complete:

60.1 Data and authoring

- target coverage meets profile requirements;
- accepted region is defined and reviewed;
- hard negatives are present;
- required and ignored regions are valid;
- unsupported conditions are documented;
- dataset partitions are locked and hashed.

60.2 Build

- reconstruction health passes;
- target references cover accepted poses;
- negative index includes strongest confusers;
- package builds reproducibly;
- package integrity verification passes;
- licenses permit intended deployment.

60.3 Verification

- all mandatory gates are enabled;
- no weighted-score bypass exists;
- positive test requirements pass;
- negative test requirements pass;
- no confirmed false accept exists in locked tests;
- guided retry achieves the required second-attempt success;
- stale-stage and duplicate-event tests pass.

60.4 Experience

- Player guidance is story-appropriate;

- Captain fallback works;
- reduced-motion and alternate input behavior work;
- Companion absence has a clear fallback;
- cinematic event is previewed and idempotent.

60.5 Safety and privacy

- capture indicator is visible;
- default runtime retention is off;
- pause and stop controls work;
- no prohibited integration is present;
- package and event authentication pass;
- cloud behavior, if used, is disclosed.

60.6 Regression

- previous certified test corpus passes under the new model bundle;
- performance budgets pass on representative hardware;
- older published stories still resolve their pinned package versions;
- rollback path is confirmed.

Chapter 61

Usability Research and Creator Onboarding

The system SHOULD be tested with creators who do not understand computer vision. A successful onboarding test is not “the engineer could use it after reading this specification.” That outcome is generally obtainable from a terminal and several regrettable evenings.

The onboarding experience SHOULD include:

1. a sample waypoint with prerecorded footage;
2. a guided explanation of target coverage;
3. an example of a hard negative and why it matters;
4. a practice build;
5. a positive and negative live test;
6. explanation of reliability grades;
7. publication and story attachment.

Usability metrics SHOULD include:

- time to first successful draft;
- number of abandoned recording sessions;
- number of creator errors requiring technical support;
- comprehension of “similar wrong places”;
- percentage of recommended remediation completed correctly;
- successful attachment to a story block;
- confidence in the reliability report.

Chapter 62

Multiplayer and Multi-Device Considerations

The baseline assumes one Player scan source per verification attempt. The architecture SHOULD allow future support for:

- multiple Players with independent Companion instances;
- one designated scanning Player;
- crew-consensus events;
- Captain selecting which Player attempt controls the checkpoint;
- a Player using a second monitor while the web experience runs on a phone or tablet;
- remote Captain operation.

A story binding MUST define whether:

- any Player may verify;
- only an assigned Player may verify;
- all Players must verify;
- the first valid attempt wins;
- attempts are shared or private.

The Companion protocol MUST not assume the Studio, Player page, and game always run on the same display, although the first implementation may require the Companion and game to share a Windows machine.

Chapter 63

Implementation Guardrails for Codex

Codex MUST NOT:

- create a hidden developer-only waypoint workflow as the permanent solution;
- hard-code the pilot location into the verifier;
- store model-specific fields directly in story blocks;
- expose raw threshold controls in Guided mode;
- silently retain runtime frames;
- treat all failures as NOT_AT_TARGET;
- enable automatic mode before certification;
- allow a model update to rewrite published package behavior without versioning;
- bypass negative testing because target tests pass;
- remove Captain override as “technical debt”;
- introduce an in-game overlay for convenience;
- turn cloud processing into an undocumented requirement;
- replace the three-state outcome with a binary result;
- return success from one frame;
- classify the whole game world when only one waypoint is active;
- invent a new architecture without an ADR.

Codex SHOULD:

- implement vertical slices with visible UI and automated tests;
- add migration-safe schemas;
- create fixtures and replayable test scans;
- isolate model adapters;
- document every new package field;
- keep creator copy friendly and technical logs expandable;
- preserve local-first operation;
- surface partial build and test findings early;
- use feature flags for promotion from shadow to automatic mode;
- provide deterministic failure codes and remediation.

Appendix A: Example Waypoint Manifest

```
{
  "schemaVersion": 1,
  "waypointId": "vp_01KSBMR7",
  "version": 3,
  "name": "Sailor's Bounty Message Rock",
  "type": "exact_landmark",
  "verificationProfile": "story_critical",
  "status": "captain_confirmed",
  "interaction": {
    "mode": "hold_to_scan",
    "durationMs": 5000,
    "sampleFramesPerSecond": 10,
    "minimumUsableFrames": 8,
    "maximumRetainedFrames": 15,
    "retryCooldownMs": 2000
  },
  "acceptedPose": {
    "volumeType": "polygonal_prism",
    "minimumDistanceMeters": 1.5,
    "maximumDistanceMeters": 7.0,
    "maximumFacingAngleDegrees": 55,
    "minimumTargetVisibility": 0.45,
    "requiredRegions": ["rock_upper_crack", "curved_tree", "upper_ridge"],
    "forbiddenRegions": ["behind_target", "waterline"]
  },
  "verification": {
    "requirePlaceRecognition": true,
    "requireHardNegativeMargin": true,
    "requireLocalGeometry": true,
    "requirePoseLocalization": true,
    "requireSpatialCoverage": true,
    "requireTemporalConsistency": true,
    "requireTargetVisibility": true,
    "ambiguityVeto": true
  },
  "runtimeAssets": {
    "globalDescriptorIndex": "descriptors/global.bin",
```

```
    "localFeatureStore": "features/local.bin",
    "sparseMap": "map/sparse.bin",
    "negativeIndex": "negatives/index.bin",
    "masks": "masks/regions.bin"
  },
  "playerGuidance": {
    "prompt": "Hold Inspect Surroundings and slowly search the area.",
    "tooFast": "The compass cannot settle. Move more slowly.",
    "tooClose": "Step back and keep more of the landmark in view.",
    "uncertain": "The compass stirs, but the signal remains unclear.",
    "negative": "The compass remains silent.",
    "verified": "The compass locks into place."
  },
  "fallback": {
    "captainApprovalAvailable": true,
    "manualPresentationAvailable": true
  },
  "compatibility": {
    "minimumCompanionVersion": "0.8.0",
    "modelBundle": "vision-runtime-2026.07",
    "supportedFov": [70, 100],
    "supportedAspectRatios": ["16:9", "16:10", "21:9"]
  },
  "certification": {
    "grade": "good",
    "positiveAttempts": 84,
    "firstScanSuccessRate": 0.93,
    "successAfterOneRetry": 0.99,
    "negativeAttempts": 3100,
    "observedFalseAccepts": 0,
    "reportId": "cert_01K..."
  }
}
```

Chapter 65

Appendix B: Verification Pseudocode

```
from dataclasses import dataclass
from enum import Enum

class Result(Enum):
    VERIFIED = "VERIFIED"
    INSUFFICIENT = "INSUFFICIENT_VISUAL_EVIDENCE"
    NOT_AT_TARGET = "NOT_AT_TARGET"

@dataclass(frozen=True)
class AttemptResult:
    result: Result
    guidance_code: str
    evidence_summary: dict


def verify_scan(scan, waypoint) -> AttemptResult:
    assert scan.waypoint_id == waypoint.id
    assert scan.waypoint_version == waypoint.version

    frames = quality_filter(scan.frames, waypoint.quality_profile)
    if len(frames) < waypoint.minimum_usable_frames:
        return AttemptResult(Result.INSUFFICIENT, "capture_quality", {})

    retrieval = retrieve_target_and_negatives(frames, waypoint)
    if retrieval.target_below_minimum:
        return AttemptResult(Result.NOT_AT_TARGET, "target_not_found", retrieval.summary
    if retrieval.ambiguous_or_negative_veto:
        return AttemptResult(Result.NOT_AT_TARGET, "ambiguous_location", retrieval.summa

    matches = run_primary_local_matcher(frames, retrieval.references, waypoint)
    if matches.insufficient:
        secondary = run_secondary_matcher_if_required(frames, retrieval.references, waypo
        if secondary.insufficient_or_contradictory:
            return AttemptResult(Result.INSUFFICIENT, "insufficient_local_detail", secon

    geometry = verify_geometry(matches, waypoint)
    if not geometry.valid:
```

```
        return AttemptResult(Result.NOT_AT_TARGET, "geometry_failed", geometry.summary()

    poses = localize_frames(frames, geometry, waypoint.sparse_map)
    if poses.too_few_valid:
        return AttemptResult(Result.INSUFFICIENT, "pose_unresolved", poses.summary())
    if not poses.inside_accepted_volume:
        return AttemptResult(Result.NOT_AT_TARGET, "outside_accepted_region", poses.summa

    if not spatial_coverage_passes(matches, waypoint.required_regions):
        return AttemptResult(Result.INSUFFICIENT, "missing_required_context", {})

    sequence = evaluate_temporal_consistency(poses, frames)
    if not sequence.valid:
        return AttemptResult(Result.INSUFFICIENT, "sequence_inconsistent", sequence.summa

    special = run_checkpoint_specific_rules(frames, poses, waypoint)
    if special.outcome == "insufficient":
        return AttemptResult(Result.INSUFFICIENT, special.guidance_code, special.summary
    if special.outcome == "negative":
        return AttemptResult(Result.NOT_AT_TARGET, special.guidance_code, special.summary

    return AttemptResult(Result.VERIFIED, "verified", build_summary(
        retrieval, geometry, poses, sequence, special
    ))
```

Appendix C: Example Studio Guidance

66.1 Capture guidance

- “Move slowly around the landmark while keeping it visible.”
- “Capture more of the right side.”
- “Step farther back so the surrounding ridge is visible.”
- “This view is already well covered.”
- “The current view is too blurry. Pause and try again.”
- “Record one more close-range view.”
- “Excellent. Target coverage is sufficient.”

66.2 Negative guidance

- “Record a nearby object that could be mistaken for the target.”
- “Test from just outside the allowed player area.”
- “Record the opposite side of the landmark.”
- “The western rock is the strongest known confuser. Add more views of both locations.”

66.3 Reliability remediation

- “The target and a wrong location are too similar. Require the curved tree and ridge in the scan.”
- “Nighttime verification is not yet supported. Record additional night views or publish with a daytime limitation.”
- “The accepted region is too broad for the available evidence. Reduce it or capture more valid positions.”

Appendix D: Pilot Waypoint Test Matrix

Test ID	Category	Scenario	Expected result
P-001	Positive	ideal front view	VERIFIED
P-002	Positive	valid left approach	VERIFIED
P-003	Positive	valid right approach	VERIFIED
P-004	Positive	close valid boundary	VERIFIED or guided retry then VERIFIED
P-005	Positive	far valid boundary	VERIFIED or guided retry then VERIFIED
P-006	Positive	alternate FOV	VERIFIED
P-007	Positive	night, supported	VERIFIED
P-008	Positive	brief player obstruction	sufficient frames still VERIFIED
Q-001	Quality	rapid camera movement	INSUFFICIENT
Q-002	Quality	menu covers scene	INSUFFICIENT
Q-003	Quality	mostly sky or water	INSUFFICIENT
N-001	Negative	nearest similar rock	NOT_AT_TARGET
N-002	Negative	correct island, wrong beach	NOT_AT_TARGET
N-003	Negative	target viewed from forbidden rear	NOT_AT_TARGET
N-004	Negative	just outside accepted region	NOT_AT_TARGET
N-005	Negative	correct region facing away	NOT_AT_TARGET or INSUFFICIENT, never VERIFIED
N-006	Negative	similar distant island	NOT_AT_TARGET
S-001	Sequence	one isolated strong frame	not VERIFIED

Test ID	Category	Scenario	Expected result
S-002	Sequence	coherent slow sweep	eligible to verify
S-003	Sequence	impossible pose jumps	INSUFFICIENT or NOT_AT_TARGET
I-001	Integrity	wrong package version	blocked
I-002	Integrity	duplicate success event	story advances once
I-003	Integrity	tampered package	rejected
F-001	Fallback	legitimate false negative	Captain can approve and label

Chapter 68

Appendix E: Source References

The following references support candidate technology choices and safety assumptions. They are informative references; this governing specification controls product behavior.

- **[R1]** Microsoft, *Windows.Graphics.Capture Namespace*. Secure application-window and display capture through Windows APIs. https://learn.microsoft.com/en-us/uwp/api/windows.graphics.capture
- **[R2]** CVG, *Hierarchical-Localization (HLoc)*. Modular six-degree-of-freedom visual localization using retrieval and feature matching. https://github.com/cvg/Hierarchical-Localization
- **[R3]** COLMAP, *Structure-from-Motion and Multi-View Stereo*. https://colmap.github.io/
- **[R4]** COLMAP, *PyCOLMAP documentation*. https://colmap.github.io/pycolmap/pycolmap.html
- **[R5]** Meta AI, *DINOv2*. General-purpose visual features. https://github.com/facebookresearch/
- **[R6]** Izquierdo and Civera, *SALAD: Optimal Transport Aggregation for Visual Place Recognition*. https://github.com/serizba/salad
- **[R7]** Keetha et al., *AnyLoc: Towards Universal Visual Place Recognition*. https://anyloc.github.
- **[R8]** Lindenberger et al., *LightGlue*. Sparse local feature matching. https://github.com/cvg/Lig
- **[R9]** ZJU3DV, *EfficientLoFTR* and *LoFTR*. Semi-dense and detector-free local feature matching. https://github.com/zju3dv/EfficientLoFTR and https://github.com/zju3dv/LoFTR
- **[R10]** Microsoft, *ONNX Runtime Execution Providers*. https://onnxruntime.ai/docs/execution-providers/
- **[R11]** Microsoft, *Install ONNX Runtime*. Windows, CUDA, WinML, and provider guidance. https://onnxruntime.ai/docs/install/
- **[R12]** OpenCV, *Feature Matching and Homography*. https://docs.opencv.org/4.x/d1/de0/tuto
- **[R13]** Sea of Thieves Support, *Enforcement Policy Updates*. Third-party visual modification policy. https://support.seaofthieves.com/articles/24643308439314-Support-Enforcement-Policy-Updates

Appendix F: Governing Summary for Codex

Codex MUST preserve the following truths:

1. Build a reusable Studio tool, not one-off waypoint code.
2. Vision Waypoints are first-class, versioned assets.
3. The default creator workflow is guided and nontechnical.
4. The player performs a short multi-frame scan, not a one-frame classification.
5. Current story context narrows the search but is not proof.
6. Global similarity retrieves candidates; it does not unlock the story.
7. Known similar wrong places are explicitly recorded and compared.
8. Local geometry and 3D camera pose are central to viewpoint tolerance and false-positive prevention.
9. Evidence must be spatially distributed and temporally consistent.
10. Ambiguity produces no reveal.
11. Guided retry protects legitimate players from brittle false negatives.
12. Mandatory gates cannot be replaced by a weighted confidence average.
13. The verifier emits an event; the story engine controls presentation and progression.
14. Captain override, shadow mode, truth labeling, and demotion are required.
15. Runtime is local-first, visible, private, and nonretaining by default.
16. The game process remains untouched; no injection, memory reading, file modification, automation, or in-game overlay.
17. Published waypoint versions are immutable and test-certified.
18. Unsafe waypoints cannot publish for automatic progression.
19. Future live external AR uses the same verification assets and event contracts.
20. Any architectural departure requires an ADR rather than a creative surprise hidden in a commit.

Final Governing Statement

The product succeeds when a creator can say, “The player must stand near this rock and look at it before the hidden message appears,” and the Studio handles capture, mapping, confusers, calibration, testing, packaging, runtime proof, guided retry, and cinematic triggering without requiring code. The machinery may be technically formidable. The interface must make it feel inevitable.